. . . for Solving and Optimizing Engineering Models

Sage
User's Guide

David Gedeon

Sage v5 Edition

Gedeon Associates
16922 South Canaan Road
Athens, OH 45701

August 10, 2007

Copyright © 1999–2006 by Gedeon Associates
Cover clipart by Image Club Graphics Inc.

Typeset in TEX with pcTEX and LATEX software

Contents

I Getting Started **1**

1 Installation **3**
- 1.1 Computer Requirements . . . 3
- 1.2 Installing . . . 3
- 1.3 Un-Installing . . . 3
- 1.4 Files . . . 4
- 1.5 New in Version 5 . . . 4

2 Overview **5**
- 2.1 What is Sage . . . 5
- 2.2 What are Models . . . 6
  - 2.2.1 Models as Trees . . . 6
  - 2.2.2 Models as Interconnected Systems . . . 6
- 2.3 Numerical Input and Output . . . 6
- 2.4 Solving, Mapping and Optimizing . . . 7
- 2.5 Model-Class Documentation . . . 7

3 Exploring a Gizmo **9**

II Reference **11**

4 Menu Commands **13**
- 4.1 File . . . 13
  - 4.1.1 File|New . . . 13
  - 4.1.2 File|Open . . . 13
  - 4.1.3 File|Save . . . 13
  - 4.1.4 File|Save As . . . 13
  - 4.1.5 File|Save Listing . . . 14
  - 4.1.6 File|Print Listing . . . 14

4.1.7 File|Listing Preview . . . 14
4.1.8 File|Save Solution Grid . . . 14
4.1.9 File|Save Tagged Variables . . . 14
4.2 Display . . . 14
4.2.1 Display|Show Window . . . 14
4.2.2 Display|Add Page . . . 14
4.2.3 Display|Remove Page . . . 15
4.2.4 Display|Remove All . . . 15
4.2.5 Display|Print Display . . . 15
4.3 Edit . . . 15
4.3.1 Edit|Show Window . . . 15
4.3.2 Edit|Up Connector . . . 15
4.3.3 Edit|Down Connector . . . 15
4.3.4 Edit|Cut Model . . . 15
4.3.5 Edit|Copy Model . . . 15
4.3.6 Edit|Paste Model . . . 16
4.3.7 Edit|Delete Model . . . 16
4.3.8 Edit|Print Form . . . 16
4.4 Scan . . . 16
4.4.1 Scan|Input Variables . . . 16
4.4.2 Scan|Mapped Variables . . . 16
4.4.3 Scan|Optimized Variables . . . 16
4.4.4 Scan|Constraints . . . 16
4.4.5 Scan|Comments . . . 16
4.5 Specify . . . 17
4.5.1 Specify|Input Variables . . . 17
4.5.2 Specify|User Variables . . . 17
4.5.3 Specify|Mapped Variables . . . 17
4.5.4 Specify|Optimized Variables . . . 17
4.5.5 Specify|Constraints . . . 17
4.5.6 Specify|Objective Function . . . 17
4.5.7 Specify|Component Name . . . 17
4.5.8 Specify|Commment . . . 17
4.5.9 Specify|Child-Model Order . . . 17
4.6 Process . . . 18
4.6.1 Process|Solve . . . 18
4.6.2 Process|Map . . . 18
4.6.3 Process|Optimize . . . 18
4.6.4 Process|Explore Optimization . . . 18
4.6.5 Process|Explore User Variables . . . 18
4.6.6 Process|Parse Solution . . . 18
4.6.7 Process|Parse Mapping . . . 18

4.6.8 Process|Parse Optimization . . . 19
4.6.9 Process|Reinitialize . . . 19
4.7 Options . . . 19
4.7.1 Options|Sage . . . 19
4.7.2 Options|Model Class . . . 19

5 Working with Models **21**
5.1 Data Files . . . 21
5.2 Viewing Model Structure . . . 21
5.3 Viewing Model Data . . . 22
5.3.1 Listings . . . 22
5.3.2 Display Form . . . 23
5.3.3 Solution Grid Variables . . . 23
5.4 Numerical Input . . . 24
5.5 System of Units . . . 25
5.6 Solving . . . 25
5.7 Solver Diagnostics . . . 26
5.8 User Variables . . . 27
5.8.1 Exploring User-Defined Variables . . . 28

6 Mapping **31**

7 Optimizing **33**
7.1 Sample Resonant System Optimizations . . . 33
7.2 Specifying Optimization Variables . . . 34
7.3 Specifying Constraints . . . 35
7.4 Specifying the Objective Function . . . 35
7.5 Running An Optimization . . . 36
7.6 Diagnostics . . . 38
7.6.1 Variable Step Limits . . . 38
7.6.2 Line-Search Step Reductions . . . 38
7.6.3 Ill-Conditioned Problems . . . 39
7.6.4 Multiple Extreme Points . . . 40
7.6.5 Normalization Values . . . 40

8 Editing Model Structure **43**
8.1 Basic Operations . . . 44
8.1.1 Activating Components . . . 44
8.1.2 Positioning Icons . . . 44
8.1.3 Navigating the Model Tree . . . 44
8.2 Component Palette . . . 44
8.3 Connections . . . 45

8.4 Changing Connector Level 45
8.5 Cut and Pasting Model Components 46
8.5.1 Copy 46
8.5.2 Cut 47
8.5.3 Paste 47

9 Variable Types **49**
9.1 Integers 49
9.2 Reals 49
9.3 Complex 49
9.4 Phasors 50
9.5 Fourier Series 51
9.5.1 Discrete Representations 53
9.6 Data Pairs 54
9.7 Cubic Splines 55
9.8 Enumerated 56

10 Entering Algebraic Expressions **59**
10.1 Ground Rules 61
10.2 Identifiers 61
10.3 Operators 61
10.4 Constants 62
10.5 Built-In Functions 62
10.6 Model-Specific Functions 63
10.7 Referenceable Variables and Qualifiers 64
10.8 Variable Scope 64

11 Specifying Drivable Variables **67**

12 Resonant-System Model Class **69**
12.1 Boundary Connections 69
12.2 Moving-Part Attachments 70
12.3 Moving-Part Variables 72
12.4 Generic Springs 72
12.5 Generic Dampers 73
12.6 Reciprocating Masses 73
12.7 Constrained Pistons 74
12.8 Relative Moving Parts 75
12.9 Relative Springs 76
12.10Relative Dampers 76

13 Behind the Scenes **77**
13.1 Smart Variables 77
13.2 Smart Models 78
13.3 Grids 78
13.3.1 Spatial Grids 79
13.3.2 Time Grids 81
13.4 Connecting Things Together 82
13.5 Solving 83
13.6 Optimizing 84
13.7 Delphi Spoken Here 85

Bibliography **87**

Part I

Getting Started

Chapter 1

Installation

1.1 Computer Requirements

Sage v4 runs under Windows platforms 98, NT, or XP. A high resolution (17 in) display monitor is convenient for editing complex models.

1.2 Installing

The correct installation procedure will update the Windows Registry and ensure that you can easily un-install your application later.

Normal Installation Insert the distribution CD ROM into the appropriate drive and use Add/Remove Programs which is located in the Control Panel. You can activate the control panel through the Start | Settings menu.

Alternative Installation Run `setup.exe` on the distribution CD or as transmitted by e-mail by double-clicking in Windows Explorer or the equivalent.

Then follow the on-screen instructions.

1.3 Un-Installing

The automatic un-installing process will remove all installed files and update the Windows Registry. Files created by you, either knowingly (data files) or invisibly (program initialization files) will remain in place. So afterwards, you might want to use Windows Explorer to manually delete files from the installation directory or whatever directory you stored data files in.

Removal Activate Add/Remove Programs in the Control Panel, select the Sage application from the listbox and follow the prompts.

1.4 Files

The default installation directory is c:\Program Files\Gedeon\Sage 4. Within the installation directory are a number of subdirectories. The first three listed below are part of the normal executable software distribution. The last two are part of the DLL or source-code distribution, licensed separately.

\Apps Contains executable program files and sample data files in subdirecties [ModelClass]\Bin and [ModelClass]\Data, where [ModelClass] refers to Gizmo, Stirling, etc. You can and should store your own data files in whatever directory you choose. For example, you might use a main directory ..\Sagework, with subdirectories as needed to keep things organized.

\Docs contains document files (generally in Adobe Acrobat pdf format) including manuals, technical notes, source-code instructions, the Sage upgrade history, and so forth.

\Help Contains the on-line help file.

\DLL Contains dynamic-link libraries and documentation in the [ModelClass] subdirectories.

\Source Contains source code commonn to all Sage applications in the Dialogs, Models, and Units subdirectories and for particular model classes in the SageApps\[ModelClass] subdirectories.

Each Sage application remembers from session to session such things as window dimensions, scroll positions, disk directories. Some of this information is stored in an application-specific file named gizmo.ini, stirling.ini, etc. Some is stored in a project-specific file named something like AName.gin, AName.sin, ..., where AName is the name of your input file and .gin stands for "gizmo initialization", .sin stands for stirling initialization", and so forth. Such files appear automatically in the directory where the executable program is stored or the directory where your data file is stored. The application-specific file is updated whenever the program closes. The project-specific file when you save the data file. Both are regenerated automatically with default values if they are lost.

1.5 New in Version 5

There have been many improvements to the overall Sage interface and individual model classes over the years. An exhaustive list is in the file UpgradeHistory.pdf in the Docs\UpgradeHistory subdirectory under the installation directory (*see* above).

Chapter 2

Overview

2.1 What is Sage

Sage is a graphical interface that supports simulation and optimization of an underlying class of engineering models. The underlying model class represents something like a spring-mass-damper resonant system, a stirling-cycle machine, or anything else that has been properly coded to work with Sage.

The model classes of Sage are not just fixed-geometry models. Each may contain an unlimited number of variations or instances. A model instance, or just plain *model* for short, is a particular collection of component building blocks, connected and assembled in a particular way, with particular data values, forming a complete system representing whatever it is you are trying to simulate. In other words, you don't just add numerical data values within the confines of a presumed geometry. You may modify the geometry too. Each particular instance of a given model class resides in its own disk file with a unique name but common file extension (such as .stl for stirling models).

Each model class comes with its own executable file for dealing with its own instances. The resonant-system model class (gizmo.exe) is common to all Sage distributions. Other model classes (stirling.exe, etc.) are distribution dependent. Running a model-class executable file brings up the common Sage graphical interface which allows you to:

- create new or read existing model files
- enter numerical data
- edit model geometry
- specify optimization problems
- solve, map or optimize the model
- view, save or print a listing

These functions are all controlled by menu commands.

2.2 What are Models

Models are more than the sum of their component building blocks. The way the components are organized and connected together is important too.

2.2.1 Models as Trees

Within Sage, model components are organized logically in a hierarchical tree structure. For example, the root model-component of a stirling machine contains a number of sub-components representing pistons, heat exchangers, and the like. These sub-components may themselves contain sub-sub-components. And so forth. The natural way to organize this in terms of child (sub) components branching off of their parent components — as trees in computer-science parlance — not unlike the directory structure on your hard drive.

The tree-structured point of view is especially convenient for organizing a model's disk file or output listing. It does not tell us much, however, about the boundary-interconnections among model components, which are crucial to understanding the functioning of the model as a whole.

2.2.2 Models as Interconnected Systems

An alternate way to present models is through their boundary interconnections, which are the abstractions by which quantities like fluid flow, force, heat flux, etc., pass from one model component to another. A special form, known as the *edit* form, presents the model from this point of view. In the edit form, each model component is represented by an icon, with sibling components (belonging to a common parent component) grouped on the same page of the form. Boundary connections among components are indicated graphically by matching numbered arrows attached to the individual model components. In this way it is possible to understand the physical connections among components. An analogy would be this: A catalog of parts, even if tree-structured, tells us little about how an automobile works. We also need to know that the wheels are connected to the engine through clutch, gearbox and differential, before we begin to understand the whole machine. So it is with Sage. To understand your model you must take some time to delve through its interconnections.

2.3 Numerical Input and Output

Model components are self-contained entities. As such they manage their own inputs and outputs. Continuing with the automobile analogy: If you want to know what a wheel is doing, ask the wheel.

In Sage, if you want to specify input data for a model component, you do so directly within that model component. And if you want to find the output for a model component, you look within that same component. One ramification of this is that output listings are organized differently than you may be used to.

Instead of finding all similar quantities from the whole model listed together, you find a sequence of component sub-listings following each other in hierarchical order. A table of contents at the beginning makes it easy to navigate through the listing. Once you get the hang of it you will find it quite easy to home in on a particular component of interest and ignore the rest.

2.4 Solving, Mapping and Optimizing

An important thing to do with models is solve them. After you modify a model's numerical inputs, some of its numerical outputs may no longer be valid. This is because models are defined in terms of implicit relationships among variables which must be iteratively solved. Solving is a menu activated process that brings numerical outputs back into sync for the whole model hierarchy simultaneously.

You can also map your model, another menu-activated process available after you have selected a number of input variables to be automatically stepped over a range of values. The stepping sequence is that which would be produced by a nested loop structure. After each step, the model is automatically solved and selected outputs are stored in a disk file for later inspection. More details on mapping are in chapter 6.

Yet another menu-activated process is optimization, which is what you do after you have specified an optimization problem — involving optimized variables, constraints and an objective function. Unlike mapping, which is an exhaustive investigation of a broad area, an optimization is more like a locally-guided walk to the top of a hill. At each step of the way the model is solved and selected outputs are stored in a disk file. More details on optimization are in chapter 7

You can find out more about what's going on behind the scenes during solving and optimizing in chapter 13.

2.5 Model-Class Documentation

As you work with specific model classes, you may find that the general information in this manual is not helping you much with the exact meaning of some model component's input and output variables or its mathematical underpinnings. For such information you will have to refer to the reference manual provided for the specific model class you are working with.

Chapter 3

Exploring a Gizmo

A good way to orient yourself with Sage is to play around with a simple resonant system, or gizmo for short. To get started, click on the icon labeled *resonant system modeler* in the Sage program group. Once the program is loaded you may either create a new file from scratch or modify any of the *.giz files, which are distributed with Sage.

Presuming you have some windows experience, you will already understand a good bit of the Sage interface. Go ahead and experiment.

Your objective should be to put together or modify a resonant system comprising some combination of springs, dampers, reciprocating masses, which are the basic gizmo model components. Each component comes in *phasor* and *time-ring* (time-grid) versions, corresponding to the type of its solution scheme. In either case the solution found will be the steady periodic solution. No transient solutions are available in gizmo-class models. Once you have assembled such a system you may specify a forcing function and solve for reciprocating mass amplitude and various forces, power flows, etc. A worthy exercise would be to set up an optimization problem whereby you solve for (optimize) frequency to maximize reciprocating-mass amplitude, given a fixed forcing function. You must include a damper in your system, though, to prevent infinite amplitude at resonance. Hint 1: Amplitude may be referenced as X.amp within the reciprocator model component. Hint 2: This very problem is solved for you in file op1.giz.

Besides chapter 4 on menu commands, you may also want to refer to subsequent chapters which discuss how you work with models in general and chapter 12 which discusses the resonant-system model class in particular.

Part II

Reference

Chapter 4

Menu Commands

The important thing to know about menu commands is that they are often keyed to the model component currently active. An active model component is the one currently selected in the display or edit form, or whose caption within the edit form is highlighted.

Summarized in this chapter are only those menu commands unique to Sage. Common Windows menu commands are not listed.

4.1 File

The file commands generally have something to do with operations involving disk files, generally for the entire model as a whole.

4.1.1 File|New

Creates a new model instance belonging to a particular model class and puts up an empty edit form ready to accept components from the palette.

4.1.2 File|Open

Opens a existing model instance file.

4.1.3 File|Save

Saves the current model instance.

4.1.4 File|Save As

Renames the current model instance before saving. Generally used when you want to make a change but save the old model too.

4.1.5 File|Save Listing

Save the model listing to a file. The model listing file is an ASCII file, not the same as the model data file.

4.1.6 File|Print Listing

Sends the model listing to the printer.

4.1.7 File|Listing Preview

Displays the model listing in a window and allows you to select various categories to appear in the listing.

4.1.8 File|Save Solution Grid

Creates a file containing detailed solution variables for any computational grids used by the model focused in the display or edit form and its connectors. Includes the grids of child model components (components that appear in lower-level windows of the edit form) and their connectors.

4.1.9 File|Save Tagged Variables

Creates a file containing all user-defined outputs selected to appear in log files. You so designate a user variable by checking the Write to Log file check-box in the user-variable input dialog. (Click on the the Specify|User Variables menu command then select a user variable from the list box and click the Change or New buttons.) Log-file tagged user-variables are indicated with an asterisk in the selection dialog list box.

4.2 Display

The display commands pertain to the display form which appears separately from the main Sage form. The display form shows textual information for selected model components.

4.2.1 Display|Show Window

Shows the display form. If showing for the first time adds the root-model page.

4.2.2 Display|Add Page

Adds to the display form a page (or pages) for an particular model component selected from a model-tree selection dialog.

4.2.3 Display|Remove Page

Removes the currently selected page from the display form. Removing a page does not affect the underlying model component that was displayed.

4.2.4 Display|Remove All

Clears all pages in the display form.

4.2.5 Display|Print Display

Prints the contents of the currently selected page in the display form.

4.3 Edit

The edit commands pertain to the edit form which appears separately from the main Sage form.

4.3.1 Edit|Show Window

Shows the edit form with the root-model page selected. You can open lower-level pages by double-clicking on the appropriate model-component icon. Or click on the page tabs. See chapter 8.

4.3.2 Edit|Up Connector

Moves up the highlighted connector arrow(s) one level in the model hierarchy toward the root, enabling you to connect it to a mate at that level. To highlight a connector arrow click on it. Hold down the shift key to highlight many.

4.3.3 Edit|Down Connector

Opposite of the `up` command.

4.3.4 Edit|Cut Model

Removes the currently selected model component from the edit form to a clip-board of sorts, as a means of deleting that component with limited undo capability. Available only if the selected component is not externally connected to any other model component. Only the most recently cut model component can be pasted back.

4.3.5 Edit|Copy Model

Copies the currently selected model component to the clip-board but does not remove it from the edit form. Useful for cloning a model component, including its child components.

4.3.6 Edit|Paste Model

Pastes a model component, including child components, from the clip board to the edit form. Multiple pastings are possible to undelete or clone model components. The paste function works for pasting a model component into the parent component originally copied from or a different parent — possibly in a different model file. This allows you to create a new model file by copying components from one or more existing model files. The only restriction is that the paste parent be capable of supporting the copy. In other words, the paste parent must contain a seed in its child-creation palette of the same class type as the component to be pasted.

4.3.7 Edit|Delete Model

Deletes the currently selected model component in the edit form. Available only if the selected component is not externally connected to any other model component.

4.3.8 Edit|Print Form

Prints the edit form as it appears on the display monitor.

4.4 Scan

For reviewing or specifying information for the whole model sub-tree beginning with the active model component. If the root model component is active then Scan includes the whole model hierarchy.

4.4.1 Scan|Input Variables

Scans and allows modification of numerical inputs.

4.4.2 Scan|Mapped Variables

Scans and allows modification of variables stepped in the map process.

4.4.3 Scan|Optimized Variables

Scans and allows modification variables solved in the optimize process.

4.4.4 Scan|Constraints

Scans and allows modification of constraints.

4.4.5 Scan|Comments

Scans and allows modification of comments.

4.5 Specify

For specifying information for the active model component only.

4.5.1 Specify|Input Variables

For numerical data input or modification.

4.5.2 Specify|User Variables

For creating and modifying special user-defined output variables. See chapter 5. In the definition dialog, checking the "write to log file" box tags the variable so it will appear in mapping or optimization log files.

4.5.3 Specify|Mapped Variables

For selecting variables to be stepped in the map process.

4.5.4 Specify|Optimized Variables

For selecting variables to be solved in the optimize process.

4.5.5 Specify|Constraints

For specifying equality or inequality constraints for the optimize process.

4.5.6 Specify|Objective Function

For specifying the objective function for the optimize process.

4.5.7 Specify|Component Name

For changing the name of a model component.

4.5.8 Specify|Commment

For entering an optional comment for a model component. A comment is a text string of arbitrary length, possibly containing multiple lines. Comments appear in the display window or listing.

4.5.9 Specify|Child-Model Order

For changing the order in which sibling model components appear in the model listing and certain dialog boxes.

4.6 Process

4.6.1 Process|Solve

Brings numerical outputs up to date with numerical inputs that may have been changed.

4.6.2 Process|Map

Carries out the mapping sequence specified by the mapped variables.

4.6.3 Process|Optimize

Carries out the optimization problem specified by the optimized variables, constraints and objective function, if any.

4.6.4 Process|Explore Optimization

Lists all optimized variables in a model, grouped by model component, with subject-to constraints listed in a parallel column. Useful for understanding the optimization structure of complicated models and for finding over-constrained or infeasible optimization specifications.

4.6.5 Process|Explore User Variables

Displays an interactive form containing a tree-structured view of all user-defined variables in the whole model. You may click on a variable identifier in the tree view to display its value and defining information or use the Find button to locate a variable anywhere in the model by matching its identifier against the one you type in the identifier box. Other buttons allow you to trace which variables reference or depend on other user-defined variables and make editing changes. A submenu that pops up after clicking the right mouse button supports copy and paste operations. *See* section 5.8.1

4.6.6 Process|Parse Solution

Parses or compiles the evaluation expressions in user-defined variables, along with other model setup tasks, without actually solving the model.

4.6.7 Process|Parse Mapping

Parses or compiles the evaluation expressions for mapped variables, without actually mapping the model.

4.6.8 Process|Parse Optimization

Parses or compiles the evaluation expressions in constraints and the objective function, along with other optimization setup tasks, without actually optimizing the model.

4.6.9 Process|Reinitialize

Resets all implicitly solved model variables to their initial values. Helpful when a change to a numerical input or model structure causes the model solver to fail to converge.

4.7 Options

4.7.1 Options|Sage

Activates a self-explanatory dialog for setting options pertaining to all Sage applications.

4.7.2 Options|Model Class

Activates a self-explanatory dialog for setting options pertaining to a particular model class.

Chapter 5

Working with Models

5.1 Data Files

Use the File|New menu command to create a new model or the File|Open command to open an existing model file. Creating a new model opens an empty edit form and fills the model-component palette within the main Sage form with potential model components. See chapter 8 for what to do after that. Opening an existing model file restores a model to its numerical state and display appearance at the time the model was last saved.

You are responsible for saving a model when you have made changes to it. Use the File|Save command or the File|Save As command if you want to save it to a new file and simultaneously change the file name for subsequent Saves. It is always a good idea to save your model before making a lot of changes to it, in case you want to revert back to the old file. Otherwise, undoing model changes is a manual process. The one *undo* feature in Sage is to re-paste a deleted (cut) model component back into the edit form, but it works only for the most recently deleted component.

When you save a model file, there are actually two files saved, a model-specific file and a Windows initialization file containing the state of the model Windows interface. The files have the same name but different file extensions, such as *.giz and *.gin for gizmo files. Other extensions are used for other model classes. You will normally not be aware of the Windows initialization file and it will be regenerated if lost.

5.2 Viewing Model Structure

The edit form, displayed with the Edit|Show Window menu command, shows the child model components within a given parent model component, along with the connections among them. If the parent component is the root-level component, you are looking at the whole model at the highest level of abstraction. The name of the current parent component is on the selected tab at the bottom of

the form. You can move up and down in the parent-child hierarchy in one of two ways: Double-clicking on a component icon in the window will activate that child as the new parent component. Or clicking on a tab will change that model component to the parent component. The tabs are designed to hold only one pathway (from root to terminal component) of the model tree at a time.

Keep in mind that many menu commands are keyed to the model component that is currently active. Within the edit form, the active model component is the one whose icon caption bar is highlighted. Or, if no component icons are highlighted, it is the parent model component itself. Highlighting a model component is just a matter of clicking on its icon. Click in the form client area between child-component icons to activate the parent component. Or click on the parent's tab.

There are two purposes for the edit form. The first is passive display of the existing model structure to help you understand what the model is all about. You may print the edit form as it appears on your display monitor, with the Edit|Print command. The second purpose is to support interactive modification of the model structure. This is the subject of chapter 8.

5.3 Viewing Model Data

Proceeding from the general to the specific, you may view your model's data with listings, display-form pages or solution-grid files.

5.3.1 Listings

The format in which model files are stored on disk is a special binary code understood by Sage alone. It is not appropriate for human readable hard copy. For readable copy, you will need an output listing which you may either preview, print or save-to-file using menu sub-commands Listing Preview, Print Listing or Save Listing under the File main menu item. An output listing contains current numerical inputs and outputs for the entire model hierarchy. There is a table-of-contents header at the beginning of an output listing which numbers its various sections (one for each model component) using a notation that conveys the model-tree hierarchy. Using this identification number it is easy to scan through the listing to the location of the model-component you are interested in.

You can select among various categories of display using the File|Listing Preview command, then checking the appropriate boxes at the top of the preview dialog. For example, to display a model's inputs only, check the *inputs* box. To display a model's optimization structure only, check the *optimized*, *constraints* and *objective function* boxes. And so forth.

The File|Print Listing command produces a non-formatted printout without margins or intelligent page breaks. For a formatted printout you should instead save the listing to a file using the File|Save Listing command. The result is an ASCII text file which you may then format and print as you see fit with your favorite word processor or text printer.

5.3.2 Display Form

The display form contains the same textual information as a listing except you include in the form only those model components you are interested in. The display for a single model component is known as a page, and each page has a corresponding tab at the bottom of the form. You display different pages by clicking on the corresponding tab. To add a new page to the form use the Display|Add Page command. To remove a page, select that page then use the Display|Remove Page command. To print a page use the Display|Print command. A display-form page contains the complete textual output for a model component. There is no way to restrict the display by categories.

The display form as a whole behaves like any other window. To make the form visible in the first place use the Display|Show Window command. After that you can close the form by clicking on the window close box in the form caption bar. Closing has the side effect of removing all the pages from the form.

Although the pages of the display form are passive, in that they do not support direct editing operations, the Scan and Specify menu items are keyed to the model component that is currently active — the one whose display page is selected.

5.3.3 Solution Grid Variables

The standard numerical outputs that appear in the listing or display form are usually single numerical values representing discrete quantities — integrals or averages, that sort of thing. Often, a model component has an underlying computational solution grid behind the outputs. You can inspect this grid via the File|Save Solution Grid command which produces an ASCII format disk file containing all the numerical solution gruds for the model component currently active in the edit form or display form and all its child model components.

You should keep in mind that not all model components contain solution grids. Generally, the highest level components (those visible in the root page of the edit form) do not. Solution grids are usually found only in lower-level components of a model class. To see which components contain grids you will have to read the documentation for the model class you are working with.

An output file may contain several grids each containing several individual state variables. The easiest way to explain this is with an actual example, such as the following which show the solution grid for a time-ring reciprocating mass of the gizmo model class:

```
Sage version 1.0 --- 5/28/96 10:05:30 AM
noname.giz
resonant system | reciprocator
position displacement grid
X: displacement (m)
-7.569E-04 -3.784E-04 3.784E-04 7.569E-04 3.784E-04 -3.784E-04
Xd: velocity (m/s)
-1.720E-20 2.471E-01 2.471E-01 1.668E-20 -2.471E-01 -2.471E-01
```

```
Xdd: acceleration (m/s2)
1.076E+02 5.378E+01 -5.378E+01 -1.076E+02 -5.378E+01 5.378E+01
```

After the header information the solution values for the three state variables of the position displacement grid are listed. The individual variable values appear as tab-delimited fields within a line or record. For the present case — a pure time grid — the entire grid of values fits in a single record, starting at time zero and equal-spaced throughout the periodic interval. For a pure space grid it would be much the same, starting at the negative domain endpoint and ending at the positive domain endpoint, also equal-spaced. For a space-time grid, there would be a sequence of records, each record corresponding to the time nodes at a fixed spatial position. The first record would consist of the time nodes at the negative domain endpoint, and so forth. In this example there is only one grid, the position displacement grid.

You can inspect a solution grid file with any text editor or word processor. Or you can open it with a spreadsheet program, such as Excel, where the tab-delimited fields should arrange themselves into neat rows and columns. Besides looking at your data, you can then plot it or operate upon it in other ways as you see fit.

5.4 Numerical Input

Once you have created a model structure or read an existing model file, you will generally want to change one or more numerical inputs.

To change a numerical input, first activate the model component in which it resides by mouse-clicking the appropriate tab in the display or edit form, or clicking the appropriate icon in the edit form. Then select Specify|Input Variables from the main menu. You will be presented with a list of input variables pertaining to that particular model component only. To change a variable value click on that variable and an appropriate input dialog will appear, geared to the format of the individual variable. It is here that you actually enter the value. Sorry, but you cannot directly edit a variable value within the display form.

Sometimes you want to change or inspect numerical inputs for more than just a single model component. This is possible with the Scan|Input Variables menu command. Using this option you are presented with a list of all model-component names requiring numerical input for the active model and its entire sub-tree. Clicking on a model-component name brings up the appropriate input-specifying dialog. Often, scanning input variables this way will be much more convenient than individually selecting model components one at a time. When you want to inspect input variables for the entire model hierarchy, just scan the root model component.

5.5 System of Units

It is possible to change the dimensional units displayed for the variables of your model. To do so, make the appropriate selections in the dimensions page of the model-class options dialog available under the Options|Model Class menu item. For example, if you change the length dimension from m (meters) to in (inches), all variables whose dimensions were previously listed as (m) are immediately converted to (in). Also affected are any variables with derived dimensions involving length. For example, variables in (m/s) are converted to (in/s).

Changing dimensional units does not affect your model's solution, nor does affect the internally-stored values for any built-in variables, which are always in SI units (International System) or dimensionless. It merely changes your model's appearance by means of a value-conversion layer of software built into the visual interface. So it is safe to change dimensions back and forth as often as required. No information will be lost.

But, changing dimensions is not entirely without subtle consequences. This is because the value-conversion layer of software also affects the internal values of any user-defined variable (section 5.8), constraint or objective function (chapter 7) whose value derives from a string expression referencing a dimensional variable. For example, if you have defined a constraint like $X \geq 0.04$, its value is based on the current dimensional value of X. Anything else would be confusing. But as a result, the constraint may be satisfied if X has units of (m) and violated if it has units of (in)! Similarly, in a mapping specification (chapter 6), the range you specify applies to the dimensional value of the mapped variable, as you would expect. So the consequences of mapping X over a range $[0.010, 0.020]$, depend on the current dimensions of X. Sage gives you the responsibility for making sure that your mapping and optimization specifications survive a dimensional change. To be safe, it is a good idea to settle on a system of units at the beginning of a project and stick with it, only changing it from time to time to compare with engineering drawings in other units or communicate with actual engineers who prefer to speak in other units.

The state of your model's dimensional units is saved in the project-specific initialization file logically paired with your model's input file (same name). This initialization file is updated whenever you save your model and read whenever you load your model. So exiting Sage without saving your model, or just reloading it, will abandon any temporary dimensional changes you might have made.

5.6 Solving

After you have modified the model structure or changed numerical inputs, you will notice that the warning *not solved* appears after the *Outputs* heading in the display form. This means you can no longer trust the values displayed. They are displayed anyway as an aid to diagnostics during the solution process.

To re-validate numerical outputs you must solve the model with the Process|Solve menu command. This command initiates an iterative process that may take a significant amount of time to finish. In some cases the solver may not even converge.

To keep you informed during the solving process, Sage puts up a status dialog which displays useful information and allows you to stop or pause the process if things are not going well. The goal is for the solver to drive the model's RMS error function (measure of how close the model's implicit function values are to zero) to some target value. When the RMS error fails to go down after a reasonable number of iterations (30–40 or so), you have trouble. You can inspect the individual components of the RMS error function and possibly diagnose problems using the techniques described in section 5.7.

Sometimes a model will not converge for a good physical reason, such as a missing boundary condition or bad initial conditions. The first and easiest thing to try is to stop the current solving process, re-initialize all variables with the Process|Reinitialize command then try again. This often works if the problem is due to remnants of a previous solution being incompatible with a drastic change in model structure or numerical input. Sage never throws away solution information unless you tell it to. It assumes previous solutions are reasonable initial conditions for subsequent solutions, which pays big dividends when you or its optimizer make only small changes to numerical inputs. If re-initializing doesn't work, it may be that the initial values are too far from the converged solution. Most model classes give you some control over this by including a number of constant inputs which are used to set initial conditions. These range from normalization values for key physical dimensions (usually in the root model component) to initial temperature distributions and the like. The idea is to set any such constants to reasonable values for your particular model-class instance. Documentation for individual model classes should offer hints about setting these constants.

If converge continues to fail, it may be that you have a physically absurd model. Helping you to understand why, is the reason Sage displays numerical outputs, even if not solved. In fact, after each solver iteration, sage updates its numerical outputs. Variables tending to zero or infinity will often give you vital clues about why your model is not converging. A good physical understanding of your model is important here. If all else fails, you may have to throw away your recent changes and resort to the last working version of your model file. You did save one, right?

5.7 Solver Diagnostics

In the event Sage's solver fails to converge and the methods discussed in section 5.6 don't help, you may want to take a look behind the scenes by checking the *solver diagnostic dialog* box under the Options|Sage menu command.

The result is a diagnostic dialog displayed after each Jacobian-matrix factorization, usually every solution iteration for a non-converging model. This dialog

allows you to inspect implicit variable values, variable steps, system function values and Jacobian matrix terms. You are looking for function values that fail to get small, variable steps that are large or hunt around back and forth, or $\frac{\partial F}{\partial V}$ values that change significantly from one iteration to the next. Since there are often upward of a thousand variables in the solution, this can be like looking for a needle in a haystack. But there are some filters available to restrict the display to manageable chunks. Keep in mind that except for V values, the values displayed in the dialog are dimensionless and normalized (scaled to a range of values on the order of one). V values are true SI dimensioned values.

In your model, each function component F is associated with an implicit variable V. For example the residual of Newton's second law of motion "mass $\times$ acceleration $-$ summation of forces", at some particular time, is the function component for the state variable Xddot (acceleration) within the reciprocating mass solution grid, at the same time. While each F is always associated with exactly one V for data-structure purposes, it generally depends on many V's — typically neighboring values in a computational grid. The partial derivatives $\frac{\partial F(\text{row})}{\partial V(\text{col})}$ are the coefficients of the Jacobian matrix, the central data structure for the iterative solution process (*see* Chapter 13). The V steps displayed in the dialog are those requested (but not necessarily taken) by the previous iteration of the nonlinear solver in its attempt to simultaneously zero all the F's.

So, for example, if in the diagnostic dialog the step dV(999) is large, or otherwise suspicious, set Col = 999 and display V(col). This should in most cases identify what V is and in which model component it is located, often enabling you immediately locate the problem. You can find out how V(999) affects various function components by selecting dF/dV(col) for display.

Function components are identified through their associated variables. So, if you want to identify F(999), you must display V(999) as before. After that, you could delve into which V's are affecting that particular F by setting Row = 999, then selecting dF(row)/dV for display. You could then delve further by identifying individually any V's that look suspicious.

If you isolate the problem to a computational grid you may want to explore that grid in detail using the File|Save Solution Grid command.

5.8 User Variables

Sometimes the numerical outputs programmed into model components may not be sufficient for your needs. You may find you are always having to add together a number of individual outputs, from one or more model components, to get the answer you are interested in. User-defined variables can help.

User-defined variables are special output variables that you yourself add to model components and define by entering algebraic expressions in terms of other variables known to that model component. They become part of the model component and appear in the display window and output listing. They may also appear in log files that accompany mappings or optimizations. Variables referenceable in the defining expression are any of a component's own input or output

variables, provided they have a numerical type, as well as any user-defined variables in its model sub-tree, provided their export level is high enough. Sounds complicated but it really isn't. You just have to be a bit organized.

For example, you may want to define a variable named MyVar whose value is (V1 + V2)/2, where V1 and V2 are names of two referenceable variables. All you do is activate the model component in which you want your variable to reside and select the Specify|User Variables menu command. A dialog will then open, allowing you to create the variable (or modify an existing one), and set its export level. A sub-dialog will prompt you for the vital information that defines the variable.

Within the sub-dialog you will have the opportunity to enter your variable's identifier name, a defining comment and an algebraic expression. In the above example the identifier name would be "MyVar" the defining comment would be anything you like (including nothing), and the algebraic expression would be "(V1 + V2)/2". The defining comment appears in the display window and output listing and is there to help you, and others who might read it, keep track of the variables purpose. More information about referenceable variables and acceptable syntax for the algebraic expression can be found in chapters 9 and 10. You may also designate your variable to appear in mapping or optimization log files by checking the *write to log file* box.

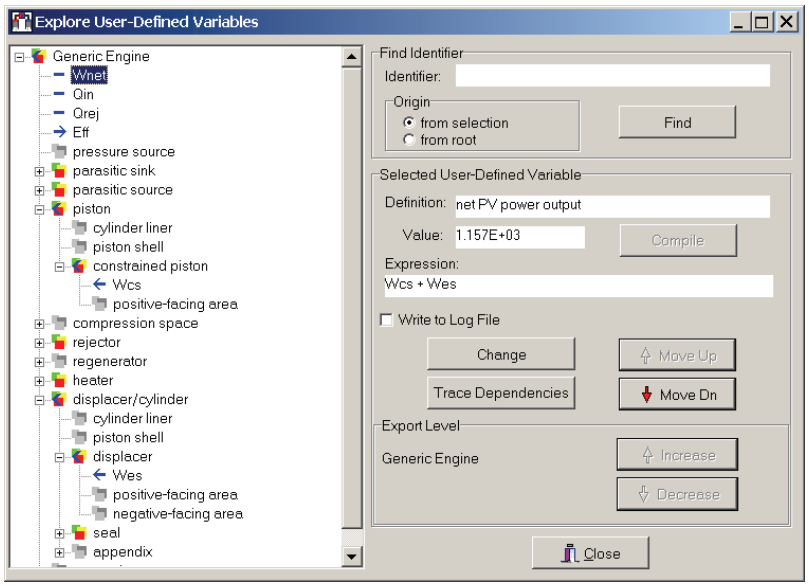
Within the main dialog that displays a list of a model component's user-defined variables, you may change the export level of any variable you select. The export level is the highest-level parent model component in which the variable will be referenceable. It is not automatically as high as possible to avoid potential name conflicts with variables in other branches of the model tree. Variables are automatically referenceable in the model component in which they are defined and any lower-level child model.

Writing algebraic expressions for user variables is a bit like programming in a typical computer language. It is possible to make grammatical syntax errors that render your expression unreadable by Sage. To see if you have done this, you may use the Process|Parse Solution menu command. Or you may just try to solve the model without taking this step. If Sage cannot parse your algebraic expression it will put up a dialog box showing your expression with the cursor at the offending location, giving you an opportunity to change it. You will find a terse comment describing the problem within the status bar at the bottom of the dialog box.

5.8.1 Exploring User-Defined Variables

The Process|Explore User Variables menu command opens up an interactive visual form where you can work with all the user-defined variables in your entire model simultaneously. The key to doing this is a tree-structured listing of model components and user-defined variables located at the left of the form, as illustrated

here:



The user-defined variables are listed as nodes under the model components in which they are defined. The displayed information, buttons and sub-menu items of the interactive form are keyed to the selected user-variable or model component in the tree view. You select a user-variable by clicking on its identifier in the tree view or by using the Find button to locate a variable anywhere in the model by matching its identifier against the one you type in the identifier box.

The Trace Dependencies button allows you to trace all the user variables that depend on or are referenced by the selected user-variable. In the example above, the user-variable Wnet is selected. It references two other user-defined variables Wcs and Wes, which are indicated by left-facing arrows in the display. It is itself referenced by another user-defined variable Eff, which is indicated by a right-facing arrow.

Several editing options are available in the panel to the right in the form. Many of these operations are also available under the Specify|User Variables menu item but they are repeated here for convenience. You can change a variables identifier, definition or expression. You can change the order in which it is listed and change its export level. The export level is the level above the parent model component in which the variable identifier may be referenced by an expression in another user-variable, constraint or whatever.

After you change a variable all user-defined variables are disabled for evaluation purposes and the value box displays "not compiled". The Compile button allows you re-parse all the variables, after which the value box once again displays numerical values.

There is also a submenu available that supports new, delete, cut, copy and paste operations. The submenu pops up by positioning the mouse cursor within the tree view then clicking the right button. Copy and paste operations use the Windows clipboard by encoding a user-variable identifier + definition + expression as a tab-delimited text string. So it is possible to copy user-variables from one model to another model running under a second Sage application. Or from a text file using a text editor into a Sage model. In the latter case it is necessary to include tab characters (ASCII code 9) between the identifier and definition, and between the definition and expression parts of the string. In order to paste a user-var that has been cut or copied it is necessary to first select the model component in the tree view into which the variable will be pasted. The same applies to the New submenu item, which inserts a new user-variable.

Chapter 6

Mapping

There are times when you may want to investigate a sequence of solutions over a discrete range of one or more input variables. This is the process of mapping. Mapping is something you might want to do to achieve a better understanding of your model's sensitivity to various inputs. But it is not the same as optimization, the topic of the next chapter.

Anyone with any programming experience will understand the sequence of solutions generated in a mapping as the result of a nested loop structure. Except no programming is required (on your part) to do a mapping in Sage. All you do is select one or more input variables to be mapped using the Specify|Mapped Variables command then run the mapping with the Process|Map command. The rest is automatic.

Prior to specifying a mapped variable you must first activate the model component containing the variable by clicking its tab in the display or edit form, or clicking on its edit-form icon. Mappable variables are selected from a list of any drivable variable within the model component as described in chapter 11 — usually a real-valued independent variable, rarely the real part of a composite variable. For each mapped variable selected, you also specify the first and last values the variable will take, the number of iterations to be made within the mapping interval and whether the variable should be mapped in equal steps or equal ratios. The total number of solutions performed in the whole mapping process is the product of the iteration counts for the individual mapped variables. Evidently, for complicated models that take a long time to solve, you will want to think carefully about how many variables to map and their iteration counts.

When mapping initiates you are prompted for the name of a disk file in which the mapping results will be stored in ASCII format. This file will contain a sequence of lines or records containing tab-delimited values for the mapped variables as well as any user-defined variables tagged for appearance in log files. You select such user-defined variables by checking the "write to log file" box in the user-variable definition dialog available under the Specify|User Variables command.

During the mapping process, solutions are generated automatically until the mapping is finished. A mapping status dialog shows you the current mapping iteration and solution progress within that iteration. Using the buttons of the status dialog you can abort the mapping at any time, after which the mapping log file is closed with the results so far. You can also view the mapping progress in the display form, as usual, while the mapping is in progress.

A sample gizmo mapping that maps the amplitude (user-variable) of a spring-mass-damper system as a function of frequency (mapped) produces a log file that looks like this:

```
Sage version 1.0 --- 5/28/96 2:30:44 PM
Mapping For: C:\SAGE\GIZ\OP1.GIZ
Omega resonant system Amplitude
5.000E+01 1.109E-02
6.000E+01 1.140E-02
7.000E+01 1.155E-02
8.000E+01 1.140E-02
9.000E+01 1.087E-02
1.000E+02 1.000E-02
```

After the two header lines comes the line identifying the mapped and user variables whose tab-delimited values appear in the remainder of the file. The variable-identification line is also tab-delimited which is more apparent if you open the map file with a spreadsheet program such as Excel. Viewing map files within a spreadsheet, or similar program, is not required but recommended for dealing with the results of complex mappings.

Chapter 7

Optimizing

The standard mathematical definition of a general nonlinear programming problem looks like this:

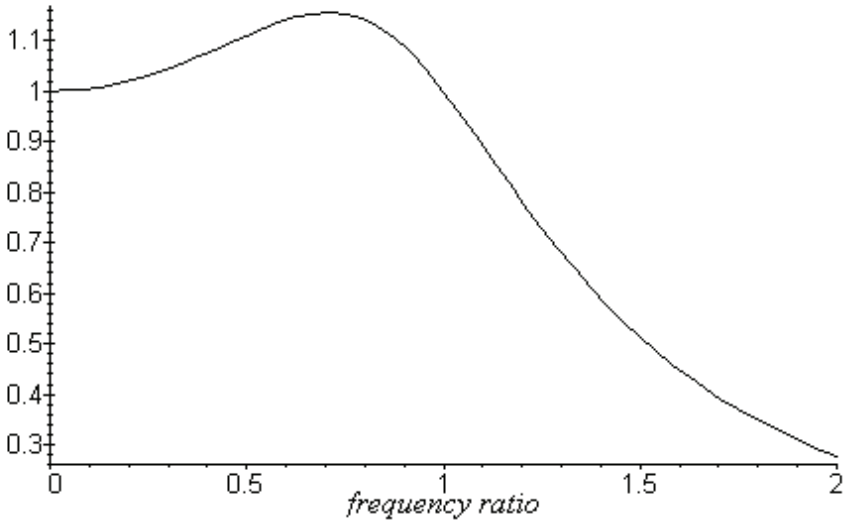
> minimize a real-valued objective function $F(\boldsymbol{x})$, where $\boldsymbol{x}$ is a vector of n real variables, subject to the equality or inequality constraints
>
> $$c_i(\boldsymbol{x}) = 0,\ i \in \mathcal{E} \quad (7.1)$$
> $$c_i(\boldsymbol{x}) \geq 0,\ i \in \mathcal{N} \quad (7.2)$$
>
> where $\mathcal{E}$ and $\mathcal{N}$ are disjoint index sets.

which may leave you cold. In Sage you deal with optimization as a natural extension of the engineering model you already understand, in terms of things you quite naturally want to do. The objective function and any constraints fall out naturally. And after you have specified a few optimization problems you might want to note that they can, after all, be cast into the above rigorous mathematical form.

7.1 Sample Resonant System Optimizations

Every mechanical engineering student learns about spring-mass-damper resonant systems of the type embodied in Sage's resonant-system model class. If you plot response amplitude in such a system when driven by a fixed-amplitude, variable-frequency forcing function you get a curve that looks something like

this:



What if you want to find the frequency that maximizes the amplitude, without actually plotting every point? This is an unconstrained optimization problem, already set up for you in sample file op1.giz. There you find Omega, (angular frequency) an input variable in the root model component, flagged as an optimization variable and objective function "maximize X.amp" specified in the reciprocator model component. Beware though of setting the damping coefficient beyond the critical-damping value ($2\sqrt{km}$). For then the maximum amplitude occurs at zero frequency.

A somewhat contrived example of a constrained optimization problem would be to optimize both frequency (Omega) and reciprocating mass (Mass) in order to maximize response amplitude, subject to the equality constraint that "Mass = Omega / 100". This problem is worked out in sample file op2.giz.

The previous two examples should be enough to get you started. The rest of this chapter is for your reference when the going gets tough. One word of warning though: always try to understand the physical principles behind your optimizations. It is much easier to specify an ill-defined problem without a solution than a well behaved one. If there is no good physical reason why an optimization should converge to a solution, then it probably won't. We must not always be blaming Sage.

7.2 Specifying Optimization Variables

Although optimization variables pertain to the model as a whole, you specify them one group at a time within each model component. To flag a variable

as an optimization variable, first activate the model component containing it (click its tab in the display or edit form, or click on its edit-form icon), then select Specify|Optimized Variables from the menu. As with mappable variables, optimizable variables are selected from a list of any drivable variable within the model component as described in chapter 11 — usually a real-valued independent variable, rarely the real part of a composite variable.

7.3 Specifying Constraints

Constraints, too, pertain to the model as a whole but are specified individually within a particular model component. To specify a constraint, first activate the model component that will contain it (click its tab in the display or edit form, or click on its edit-form icon), then select Specify|Constraints from the menu. Constraints are specified in the form $E_1 \leq E_2$, $E_1 = E_2$, $E_1 \geq E_2$, where $E1$ and $E2$ are two algebraic expressions you enter, similar to those entered for user-defined variables. More information about entering algebraic expression can be found in chapter 10.

Satisfying your constraints is the job of Sage's optimization driver. It does this by tweaking the optimization variables to force the left-hand-side expression minus the right-hand-side expression ($E_1 - E_2$) to be zero, in the case of an equality constraint, or merely non-negative or non-positive in the case of an inequality constraint. Equality constraints are always said to be *active* — meaning *in force* or *to be reckoned with*. Inequality constraints are active only when they start out violated or the optimization driver tends to violate the constraint during the course of the optimization. A useful rule of thumb is that each active constraint reduces by one the degrees of freedom available in the search domain to minimize or maximize the objective function.

Constraints are very useful things. You can invoke simple inequality constraints to keep an optimized variable in bounds — such as "X $\leq$ C" or "X $\geq$ C". Or, you can invoke more complicated constraints — such as PowerOut = 100, where PowerOut is a user-defined variable you have defined in terms of built-in outputs. You can even invoke constraints without an objective function present. However, when you do this it is necessary to specify the same number of optimization variables as active constraints. Go ahead and be liberal imposing constraints. Create as many as you feel you need. If physically sensible and reasonably linear, they are no great burden on Sage's optimizer.

7.4 Specifying the Objective Function

Like all the other elements of an optimization problem, the objective function pertains to the model as a whole but is specified within a particular model component. There can be only one objective function, which means that if you create a new one, the old one disappears. To specify the objective function, first activate the model component in which it is to appear (click its tab in the display

or edit form, or click on its edit-form icon), then select Specify|Objective Function from the menu. You will be presented with dialog containing information about the existing objective function, if any, and the means to change it. The objective function is specified in the form "minimize E" or "maximize E", where E is an algebraic expression similar to those entered for user-defined variables or constraints. More information about entering algebraic expression can be found in chapter 10.

In the objective-function dialog box, the existing objective function is displayed even if it is in another model component. Where it presently resides appears in the dialog box labeled *Currently In Model Component*. Where a newly created or changed objective function will reside appears in the dialog box labeled *New Model Component*. The component in which the objective function belongs is the one where it is most natural to reference the variables in its defining expression, usually, but not always, the root model component.

As with satisfying constraints, Sage maximizes or minimizes your objective function by tweaking optimization variables according to its built-in logic. The objective function often represents some model output such as power, heat input, efficiency — typically, something that arises as the end result of a considerable amount of computation. As such, objective functions can be highly non quadratic (the ideal) and require many iterations for Sage to minimize or maximize. On the other hand, they may be relatively simple, as in the sample problems concocted at the beginning of this chapter.

7.5 Running An Optimization

Once you have specified the optimization problem, you initiate the optimization process with the Process|Optimize menu command. When optimization begins you are prompted for the name of a disk file in which the optimization results will be stored in ASCII format. This file will contain a sequence of lines or records containing tab-delimited values for the optimized variables, constraints and objective function, as well as any user-defined variables tagged for appearance in log files. You select such user-defined variables by checking the "write to log file" box in the user-variable definition dialog available under the Specify|User Variables command.

A status dialog box then appears, giving you a blow-by-blow account of the process and an opportunity to stop or pause if the need should arise. You can use the pause option to inspect the current solution state or save the model between iterations. The status dialog itself contains a good deal of information designed to keep you somewhat informed and entertained while you are sipping your coffee, or doing whatever you do to kill time. The goal of the optimizer is to drive the so-called pseudo-Lagrangian step change to a small but negative value.

The pseudo-Lagrangian step-change is a measure of the relative change over the current step of the objective function plus a weighted sum of violated constraints. The value should start out large and negative and grow ever smaller

(but still always negative) as the optimization converge to the minimizer. For our present purposes, it is sufficient to think of the pseudo-Lagrangian as an approximation to the classical Lagrangian function of constrained optimization theory, which has an extreme point at the minimizer or maximizer.

One thing to keep in mind is that values printed in the status dialog pertain to the optimization problem at a somewhat higher level of abstraction than your original specification. The objective function and constraint violations displayed are normalized values. And if you have chosen to *maximize* rather than *minimize* your objective function, its sign is switched. This is because the optimizer always *minimizes* objective functions. Minimizing $-f$ is equivalent to maximizing f.

After some up-front work to estimate evaluation precision in the objective function and constraints and to initialize the Hessian (second-derivative matrix), an optimization is carried out as a sequence of iterations. Each iteration requires a small step of each optimization variable in order to perform numerical partial derivatives of the objective function and its constraints, followed by a line search (all variables stepped simultaneously) along a direction defined by the solution to a quadratic-programming subproblem the optimizer maintains as it goes along. You can follow this in the status dialog box labeled *stepping*. Each step requires another model solution.

If all goes well, Sage will converge to a unique solution within a reasonable period of time. Satisfying any violated constraints usually requires only a few iterations, say less than ten. After that, Sage seems to concentrate more on minimizing or maximizing the objective function, which may take a good deal longer, say up to forty iterations — maybe more in some cases. After convergence you should inspect the state of your model then save it using the File|Save command if all is well.

Sometimes, due to noise in the model, the optimizer will never reach its target pseudo-Lagrangian step change. It will achieve a small value in the range of, say, (-10^{-5}) to $(-10^{-8}$ and never get lower. This is generally good enough and you should feel free to stop the optimizer whenever you think it is done. After all, you are a good deal more intelligent than Sage's optimizer and should be able to sense when your model is as optimized as it is ever likely to get. The optimizers main advantage over you is superior diligence.

When your optimization is finished you may want to take a look at the log file. The sample op1.giz gizmo optimization mentioned earlier, produces a log file that looks like this:

```
Sage version 1.0 --- 5/28/96 3:37:22 PM
Optimization For: C:\SAGE\GIZ\OP1.GIZ
Omega resonant system Maximize X.amp
5.000E+01 1.109E-02
7.902E+01 1.143E-02
6.560E+01 1.151E-02
6.992E+01 1.155E-02
7.082E+01 1.155E-02
```

```
7.071E+01 1.155E-02
7.071E+01 1.155E-02
```

After the two header lines comes the line identifying the optimized variable and objective function whose tab-delimited values appear in the remainder of the file. The identification line is also tab-delimited which is more apparent if you open the map file with a spreadsheet program such as Excel. Viewing map files within a spreadsheet, or similar program, is not required but recommended for dealing with the results of complex optimizations.

7.6 Diagnostics

The most obvious symptom of a failing optimization is when the magnitude of the pseudo-Lagrangian step change fails to decrease, or even increases during the optimization process. There are other signs to look for that may help you pinpoint the problem.

7.6.1 Variable Step Limits

Keep an eye on the status-dialog box labeled *step limit for*. Ideally it should displays *none*, which is a sign that all is going smoothly and the optimizer's quadratic approximation to your optimization problem is reasonably valid. If it displays the name of one of your optimization variables instead, then the optimizer is having trouble with that variable. It has tried to step it to a forbidden value. If the problem does not clear up after a few iterations, it may be a good idea to stop and restart the optimization after a careful review of the current state of the optimization variables.

7.6.2 Line-Search Step Reductions

Another think to look for is two successive model solutions, instead of one, during the line search. You may have to be quick to notice this, depending on the time the solver spends solving your model. The first step of the line search is with the increment returned by the quadratic programing subproblem. If that step fails to produce the expected decrease in the pseudo-Lagrangian function, then a second, reduced, step is taken. When a failed search step happens near the beginning of an optimization, it usually means that either the Hessian (second-derivative) matrix, which is internally maintained and updated by Sage's optimizer, has not yet had time to fully evolve, or the pseudo-Lagrangian function is nowhere near quadratic. Usually the problem is self-correcting after a few iterations as the quadratic approximation begins to better fit the nonlinear pseudo-Lagrangian. If the problem does not go away it may indicate an ill-conditioned optimization as described below. Sometimes it helps to restart the optimization.

7.6.3 Ill-Conditioned Problems

When Sage absolutely fails to converge, it is probably suffering from an ill-conditioned optimization problem. You will want to be on guard for some of the following cases.

The most obvious example of ill-conditioning is a problem without a minimum or maximum. An example might be maximizing heat lift in a stirling cooler but forgetting to constrain input power. The size and power of the machine may go up forever. It is usually not difficult to avoid this problem, but sometimes the interaction of objective function and constraints is subtle. The symptom of a problem without a minimum or maximum is optimization variables drifting off to infinity or zero while the objective function keeps decreasing steadily.

A similar problem, but not as easy to spot, is the problem of weakly-determined optimization variables — subsets of variables with more degrees of freedom than required to solve the problem. Say, for example, you are optimizing a rectangular-channel heat exchanger with three pertinent physical variables: channel number, width and gap. And assume your objective function and constraints depend only on the two derived variables: hydraulic diameter and wetted perimeter. Then there would be no way you could optimize all three physical variables simultaneously. Nor could Sage. The best you could hope to do would be to optimize two physical variables, fixing the third. If Sage were to attempt to solve for all three, it would probably do its best, but wind up drifting around inconclusively. Whenever you select an optimization variable, take time to think of how the objective function and constraints depend upon it. There is no point adding a variable if it has no bearing on either. And if a variable is present solely to satisfy a constraint, be sure it is the only such variable.

The most ill-conditioned problem of all is one with infeasible or overdetermined constraints — constraints that cannot possibly be satisfied. A simple example might be a problem with the two inequality constraints "pressure $\geq$ 10.0E5" and "pressure $\leq$ 5.0E5". No one would be that foolish, but when constraints get complicated, incompatibilities become less glaring. Sage has *big* trouble when faced with such constraints. The main symptoms are very large pseudo-Lagrangian step changes but no real improvement in constraint violations or the objective function. The objective function may *increase* dramatically (wrong way) as it takes a back seat to Sage's attempt to satisfy your constraints. If you have any experience running Sage with a well-behaved problem, you will immediately recognize the symptoms of infeasible constraints.

If the optimizer feels the problem is bad enough it will terminate the optimization process with a message box containing some information about the nature of the problem. Always check for the possibility that you have more equality constraints (plus active inequality constraints) than optimization variables. Otherwise, because of small numerical differences in the linear constraint approximations, it is rare for constraints to be truly infeasible. More likely, you will see the weird behavior described earlier than any error messages.

Occasionally Sage may suggest that your inequality constraints are infeasible, even though you have specified no inequality constraints. This is not an error. It merely reflects the status of two hidden inequality constraints added to your problem by Sage's optimizer. When this happens, you should infer that your equality constraints are really the infeasible ones.

If all else fails, after 100, or so, iterations — more than enough for all foreseeable well-conditioned optimizations — Sage's optimizer is programmed to give up, displaying its reason in a message box. The exact number of iterations before this happens may be model-class specific.

7.6.4 Multiple Extreme Points

Unconstrained optimizations generally lead to a unique solution — independent of starting conditions. There are two reasons for this: First, in most engineering models, optimization variables tend to cluster themselves into weakly coupled subsets (within model components or component subtrees). And second, within each of these subsets model outputs tend to vary smoothly, or at least without multiple peaks and valleys.

Add constraints and all bets are off. Constraints, especially nonlinear inequality constraints, tend to carve up the search domain into weird shaped, possibly disjoint, regions, with multiple local extreme points potentially found on the boundaries. Moreover, these regions are in n-dimensional space (where n is usually greater than 3) and are extremely difficult to visualize.

The problem is that Sage will happily converge to any local minimum with complete disregard for a *better* local minima lurking in some other corner of the search domain. This is a problem with any gradient-based search algorithm. The only way to find multiple solutions is to restart Sage from several different points in the search domain. But the problem is, how do you pick these starting points?

The most satisfactory approach to finding a global optimum is by way of successive approximation, building intuition about your problem as you go. Start out by solving a problem with as few constraints as possible, then add constraints one by one and see what happens. Save the data files from intermediate results in case you need to return to them as future starting points. As you add constraints you will be able to develop some intuition about where that constraint drives the solution, why it does so, whether or not there might be other local extrema, and where to look for them. Eventually you will accumulate a number of good starting points from which to start Sage for the final case where all constraints are simultaneously present.

7.6.5 Normalization Values

When specifying an optimized variable, constraint or the objective function, you can change its normalization value using an edit box in the specification dialog. Actually, what you can change is the scale factor which multiplies the default

normalization value. But first, what is a normalization value and why might you need to change it?

A normalization value establishes a representative size for a *dimensioned* value, providing the necessary information to convert it to a uniform sized *dimensionless* value. A dimensioned value may be a very small number or a very large number, depending on physical units. For example, the value of a clearance seal gap might be of the order of 10^{-5} m while the value of charge pressure might be of the order 10^{6} Pa. A dimensionless value is always of uniform size, on the order of one.

The way Sage converts a dimensioned value to a dimensionless one is by dividing by the normalization value. Sage automatically maintains a default normalization value for each optimized variable, constraint or objective function. The basis for these normalization values are special input constants, usually defined in the top-level model component and usually with the suffix "norm" in the variable name, as in Lnorm. So, for example, a gap of 10^{-5} m becomes, in dimensionless form, gap / Lnorm. Normalization constants for the more-complicated expressions found in constraints and objective functions are built up as similar expressions involving basic normalization constants.

The good news is that you need not be concerned about normalization values in most cases. As long as the normalized values of all optimized variables, constraints and the objective function are within an order of magnitude or two of each other, the optimizer can successfully complete the optimization. On rare occasions, however, a normalization value can be too small or too large. If too small, then the optimizer is fooled into stepping farther than necessary when making finite-difference approximations to partial derivatives. Sometimes stepping so far that the model solution fails to converge. If the normalization value is too large, then the optimizer may not step far enough to get an accurate finite-difference result. Or it may be lulled into thinking that the optimization has converged because the objective function is changing by what it things is a very small amount.

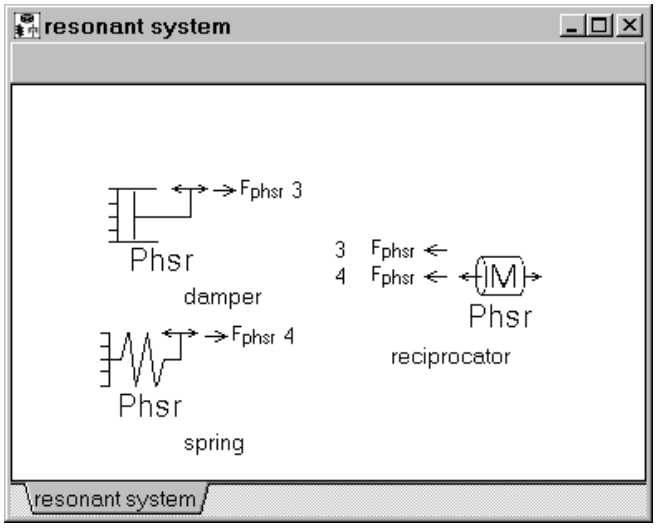
To allow you to diagnose this sort of problem, the value displays of optimized variables, constraints and the objective function (visible in display windows during optimization or after Process|Compile Optimization) include the normalization value in parenthesis, as in (norm = 1.654E-2), or some such. So, for example, if your objective is to maximize Wpv (where Wpv might be a user-defined variable for stirling-engine PV power output), you might expect trouble if Wpv is of the order 10 W, and norm is of the order 10^{4}. To remedy the problem, you would set the normalization scale factor for the objective function to 10^{-3}, using the edit box in the specification dialog. This would correct the value of norm to 10 (visible after another Process|Compile Optimization), resulting in a better conditioned optimization problem.

Chapter 8

Editing Model Structure

In Sage, you edit (create or modify) a model's structure within a graphical interface known as the edit form, separate from the main Sage form. To open the edit form use the Edit|Show Window menu command. The edit form contains a number of pages at different levels of the model hierarchy. These pages are created as explained below and selected by mouse-clicking on the tabs at the bottom of the edit form. The leftmost page always shows the child components of the root-level model component.

The edit-form root page for the resonant-system sample model phasor.giz, looks like this:



Visible in the window are graphical icons representing child model components of the root model. Attached to the child-model icons are arrows corresponding to their boundary connections. An arrow with a number beside it

indicates an active connection to another model icon within the same window. Active connections are identified by matching numbers. Arrows without numbers indicate potential connections, as yet unmade. The reason connections are indicated with numbers rather than attachment lines is to avoid cluttering up the window in complicated models.

Boundary connections can be made freely, but only among similar type connectors. Similar type connectors are indicated by the symbolic code next to the arrows. In the above example, the arrows labeled Fphsr indicate phasor forces. Phasor (complex-number) amplitudes are the way the resonant-system model class represents sinusoidally time-varying quantities.

8.1 Basic Operations

The edit form has a basic mouse interface for certain common operations.

8.1.1 Activating Components

Each model-component icon in the edit form has a caption bearing the name of the model component it belongs to. Clicking on a component icon highlights its caption bar, which activates that model component for pertinent menu operations.

8.1.2 Positioning Icons

Holding down the left mouse button while the mouse cursor is on a component icon allows you to drag the icon anywhere within the edit form. Releasing the mouse button drops the icon at its new position.

8.1.3 Navigating the Model Tree

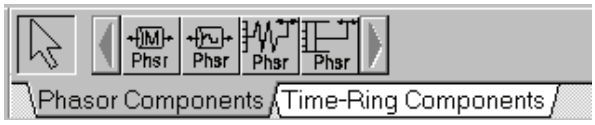
Model components can have child components, and so on for several generations. To display a component's children just double-click on its icon. This adds a new page to the edit form and a new tab at the bottom. You can re-display the parent component by double-clicking in the form's client area between icons. Or you can change pages by clicking on the tabs.

The tabs hold only a single branch pathway from the root to some lower-level component. When you change the tail of a path, by double clicking on a different component icon, the tabs corresponding to the old tail disappear.

8.2 Component Palette

When the edit form is active, a model-component palette appears in the main Sage form. This palette contains buttons representing the seeds for new model

components which can germinate in the current page of the edit form. The component palette for the previous example looks like this



The buttons of the palette are themselves arranged in tab-selected pages corresponding to different categories of model components. The available components in the palette are those which make sense in the context of the parent component, generally a small subset of the total number of components in the model class.

To create a new model component click a button in the component palette, then click again in the edit form at the position you want the component to reside. Once you've created a model component you can drag it around the edit form with the mouse, as describe above.

Considerably more difficult than the problem of creating model components is deciding which ones to create in the first place. For help with this you might want to look at the sample data files distributed with your particular model class. Or you may want to read about a particular model component in your model-class user's guide, usually a separate manual from this one. For information about resonant-system model-class components, see chapter 12 of this manual.

8.3 Connections

Making or breaking connections between model components is simply a matter of drag-and-dropping a connector to its mate.

To connect together two compatible boundary connectors, click on the first connector arrow with the left mouse button, then with the button held down position the mouse cursor on the second connector arrow and release the button. The first-clicked connector arrow establishes the type of the connection, and locks it in. The connector arrow you drag to must then be a mating arrow (opposite sense) of the same type (label).

Breaking connections is just like making them. Click on the first connector arrow with the left mouse button, then with the button held down position the mouse cursor on the mating connector arrow and release the button.

8.4 Changing Connector Level

Often, the desired mate to a boundary connector resides at another location in the model tree, not displayed on the same page of the edit form. So the two

boundary connectors are not immediately connectable. In this case it is always possible to trace up the model tree (toward the root) from each boundary-connector's model component to find two ancestor model components that appear within a common page. This is useful because, in Sage, you are allowed to move boundary connector arrows up the model hierarchy. Connector arrows so moved appear attached to the ancestor model-component icon. Once they are there they may be attached to any compatible boundary connector at the new level. By doing this for our original two star-crossed connector arrows, they can be eventually joined.

To move a boundary-connector arrow *up* the model tree (toward the root) first highlight the arrow by clicking on it then click on the Inc connector level speed button in the main Sage form (red up-directed arrow) or use the Edit|Up Connector menu command. If you hold-down the shift key, you can highlight several connector arrows at the same time, after which you can move them as a group. But you can only highlight an arrow that is not already connected.

To move a boundary-connector *down* the model tree, first highlight one (or several) disconnected arrows then click on the Dec connector level speed button in the main Sage form (red down-directed arrow) or use the Edit|Down Connector menu command.

The notions of *up* and *down* may or may not make sense to you, depending on your mental picture of a model tree. For purpose of moving connectors you might want to visualize the model tree with the root at the top and branches extending downward.

8.5 Cut and Pasting Model Components

The standard Window's conventions for cutting and pasting blocks of text in a text editor apply to model components in the edit form. Cut and copy operate on the currently selected model component and insert that component into a clipboard-like temporary storage area. The clipboard is the source for one or more subsequent paste operations.

This clipboard is not the same as the standard Window's clipboard for textual information. So you will not be able to paste into another Windows application a model component cut from Sage.

8.5.1 Copy

First highlight a model component by clicking on its icon. Then click on the Copy speed button in the main Sage form (overlapped pages) or use the Edit|Copy Model menu command. The model component, including all generations of child components, is now copied into the clipboard for subsequent pasting. Copying does not affect the existing model component so it is useful for cloning or duplicating that model component as a sibling component.

This only works for model components whose boundary connectors are not externally connected. If you want to copy a connected component first break

its boundary connections using the drag-and-drop techniques described above.

8.5.2 Cut

First highlight a model component by clicking on its icon. Then click on the Cut speed button in the main Sage form (scissors) or use the Edit|Cut Model menu command. This is just like the copy operation except the model component is deleted from the edit form. The cut operation is the only way to delete a model component.

8.5.3 Paste

Following a copy or cut operation, click on the Paste speed button in the main Sage form (clipboard) or use the Edit|Paste Model menu command to spawn a new model component in the edit form. A model component can only be pasted into the same page it was cut or copied from. The paste operation serves as a limited means of undoing a previous model deletion.

Chapter 9

Variable Types

Sage models employ a number variable types that you need to be aware of. These include the common integer and real (floating-point) variables but go on quite a bit beyond that.

9.1 Integers

Integer variables display as numerals without decimal points like this:

```
NCell           number spatial cells            2
```

Sometimes integer variable inputs are restricted to a range of values.

You may reference integer variables directly by name in user-defined algebraic expressions. They are converted to real values when you do this.

9.2 Reals

Real variables display in exponential format like this:

```
Freq            frequency (Hz)                5.500E+01
```

Sometimes real variable inputs are restricted to non-negative, strictly positive or values within some prescribed closed interval. You may enter real values in dialog boxes with or without decimal points or the E suffix.

Real variables may be referenced directly by name in user-defined algebraic expressions.

9.3 Complex

Complex variables display in an extended exponential format like this:

```
F           boundary force (N)       (-9.466, 7.633)E+01
```

The numbers in parenthesis are the real and imaginary parts and the `E+01` is the common power-of-ten multiplier. The above example is equivalent to the complex number $(-94.66, 76.33)$ or $-94.66 + 76.33i$, where i is the imaginary unit ($i = \sqrt{-1}$).

In model components with an underlying basis in linear differential equations, complex amplitudes are often used to represent sinusoidal time-varying quantities. Presuming this to be the case in the above example, physical boundary force would actually be understood as the real-part of $Fe^{i\omega t}$, or

$$\text{physical boundary force} = -94.66\cos\omega t - 76.33\sin\omega t$$

If we look at this as just the first-harmonic terms in a Fourier-series expansion $\sum(a_n\cos n\omega t + b_n\sin n\omega t)$, we note that $a_1 = \Re F$ and $b_1 = -\Im F$, where $\Re F$ and $\Im F$ are the real and imaginary parts of complex amplitude F.

Complex variables are not directly referenceable in user-defined algebraic expressions. However for a complex variable X, the following subfields are referenceable:

X.real	real part
X.imag	imaginary part
X.amp	amplitude or modulus
X.arg	argument or phase angle

9.4 Phasors

Phasors are just complex numbers displayed in polar form. A typical phasor variable would be displayed like this:

```
F           boundary force (N)       1.216E+02 cis(2.463)
```

The leading coefficient (1.216E+02 in this case) is the complex magnitude or amplitude — the length of the phasor. The number in parenthesis (2.463) is the phase angle in radians measured counterclockwise from the positive real axis. The notation "cis" reads "cosine plus i sine", where i is the complex unit. In other words, the above variable is equal to the complex number $121.6\cos(2.463) + 121.6i\sin(2.463)$, which is the same as the number in the previous example.

Under the convention that boundary force actually represents a complex amplitude for a sinusoidal time-varying quantity, we would now have

$$\text{physical boundary force} = 121.6\cos(\omega t + 2.463)$$

If we look at this as just the first-harmonic term in a Fourier-series cosine expansion $\sum(c_n\cos(n\omega t + r_n)$, we have $c_1 = |F|$ and $r_1 = \angle F$, where $|F|$ and $\angle F$ are the magnitude and phase angle of phasor amplitude F.

Phasors are indirectly referenceable in user-defined algebraic expressions through the same subfield identifiers as for complex numbers.

9.5 Fourier Series

Fourier-series variables have a multi-line display like this:

```
FTmean        x-mean temperature (K)        2.990E+02...
  ( 1.597,  0.428,  0.301)E+01 Amp
  (-2.894, -0.431,  3.142)E+00 Arg
```

The number to the right on the first line is the time-mean value and numbers in the subsequent two lines are the common-exponent amplitudes and phases of the first three harmonics in the Fourier-series cosine expansion. The above example is an abbreviation for

$$\begin{aligned} T &= 299 \\ &+ 15.97\cos(\omega t - 2.849) \\ &+ 4.28\cos(2\omega t - 0.431) \\ &+ 3.01\cos(3\omega t + 3.142) \end{aligned}$$

where t is time and ω is angular frequency. The amplitude and phase of the first harmonic in a Fourier cosine series correspond to the amplitude and phase of a complex or phasor-type variable.

An alternate, though less common, format is in terms of coefficients in a cosine-sine expansion like this

```
FTmean        x-mean temperature (K)        2.990E+02...
  (-1.529,  0.389, -0.163)E+01 Cos
  ( 0.461,  0.179, -0.253)E+01 Sin
```

which would be an abbreviation for

$$\begin{aligned} T &= 299 \\ &- 15.29\cos\omega t + 4.61\sin\omega t \\ &+ 3.89\cos 2\omega t + 1.79\sin 2\omega t \\ &- 1.63\cos 3\omega t - 2.53\sin 3\omega t \end{aligned}$$

The two representations are completely equivalent, as can be seen by applying the cosine angle-addition formula to the cosine series

$$\begin{aligned} \sum_{n=1}^{\infty} c_n\cos(n\omega t + r_n) &= \sum_{n=1}^{\infty}(c_n\cos r_n)\cos n\omega t - \sum_{n=1}^{\infty}(c_n\sin r_n)\sin n\omega t \\ &= \sum_{n=1}^{\infty} a_n\cos n\omega t + \sum_{n=1}^{\infty} b_n\sin n\omega t \end{aligned}$$

Evidently, one can convert from cosine-only to cosine-sine series using the relationships

$$\begin{aligned} a_n &= c_n \cos r_n \\ b_n &= -c_n \sin r_n \end{aligned}$$

or conversely

$$\begin{aligned} \tan r_n &= -b_n/a_n \\ c_n^2 &= a_n^2 + b_n^2 \end{aligned}$$

However, one must use caution if the need arises to shift the reference phase in an amplitude-phase expansion. Each harmonic shifts phase by a different amount. To see this let $\theta = \omega t - r_0$ denote a phase shifted time where r_0 is some fixed phase angle. Then

$$\cos(n\omega t + r_n) = \cos(n\theta + r_n + nr_0)$$

from which it is clear that the n-th harmonic in a θ cosine series is phase-shifted by an amount nr_0 compared to the original ωt series.

The number of harmonics displayed is automatic in the case of an output variable. It is generally half the number of time nodes specified in the underlying computational grid, the maximum resolvable amount according to the Nyquist sampling theorem.

For an input variable, the number of harmonics is up to the user, although higher harmonics are not resolvable, and in fact will cause aliasing errors, in any computational grid containing fewer than twice as many time nodes as the number of the harmonic. Amplitudes and phases for individual harmonics are changed via dialog-box commands.

The higher-frequency harmonics in a Fourier-series output variable should be small compared to the lower-frequency harmonics. When this is the case, it suggests that the number of time nodes in your model's time grid is sufficient to resolve the solution and all is well. If this is not the case, then you may be fooling yourself. At best, the higher-frequency harmonics are simply missing from the output. At worst, they are actually showing up as erroneous contributions to low-frequency harmonics, in which case you may be in for a rude awakening in the test cell. So keep an eye on those high-frequency coefficients and increase the time nodes in your solution grid if necessary.

Fourier-series variables are not directly referenceable in user-defined algebraic expressions. However for a Fourier-series variable X, the following subfields are referenceable:

X.mean	time-mean value
X.cos.n	nth-harmonic cosine coefficient
X.sin.n	nth-harmonic sine coefficient
X.amp.n	nth-harmonic amplitude
X.arg.n	nth-harmonic phase angle

In the last four cases, n is an integer, ranging from 1 up to the maximum available harmonic index.

You may also reference the evaluated result of a Fourier-series variable by following the variable name with an argument enclosed in parenthesis. For example, if X is a Fourier-series variable then X(0.5) returns X evaluated at $\omega t = 0.5$. The argument can be any valid expression as defined in chapter 10, not just a simple constant. The dimensional units of the argument are presumed to be the same as the current dimensional units in effect for angles (Options|Model Class dialog).

9.5.1 Discrete Representations

There are some subtle differences between Fourier-series representations of continuous functions and representations of discrete functions defined only at the nodes of a time grid. You should be aware of these in case you plan to work out the Fourier coefficients of a special function defined in terms of a table of values. The general formulation for a function $f(\tau_j)$, defined at points $\{\tau_j = 2\pi j/N : j = 0 \ldots N-1\}$ is

$$f(\tau_j) = f_m + \sum_{n=1}^{M} (a_n \cos n\tau_j + b_n \sin n\tau_j)$$

In the above formula, N corresponds to the Sage input variable NTnode and M is either $N/2$, if N is even, or $(N-1)/2$, if N is odd. Variable τ substitutes for ωt. The Fourier-series coefficient are

mean value

$$f_m = \frac{1}{N} \sum_{j=0}^{N-1} f(\tau_j)$$

cosine coefficients $n = 1..M$

$$a_n = \frac{2}{N} \sum_{j=0}^{N-1} f(\tau_j) \cos n\tau_j$$

sine coefficients $n = 1..M$

$$b_n = \frac{2}{N} \sum_{j=0}^{N-1} f(\tau_j) \sin n\tau_j$$

Except for the case N even, where a_M is half the above value and $b_M = 0$ (technically, b_M is half the above value too, however it evaluates to zero because $\sin M\tau_j$ is always zero). For Sage input variables in terms of cosine-coefficients only, the previous conversion equations can be used to convert the a_n and b_n coefficients to c_n and r_n.

The above formulation is the real equivalent of the complex discrete Fourier transform, which says for a complex-valued function $\boldsymbol{F}$

$$\boldsymbol{F}(\tau_j) = \sum_{n=0}^{N-1} \boldsymbol{c}_n e^{in\tau_j}$$

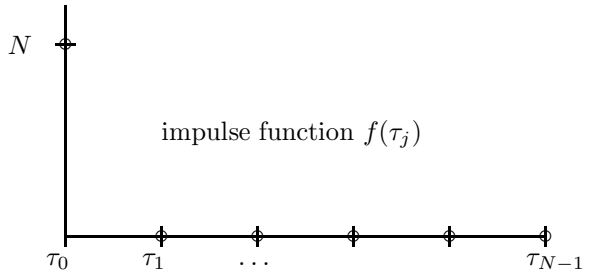
if and only if

$$\boldsymbol{c}_n = \sum_{j=0}^{N-1} \boldsymbol{F}(\tau_j) e^{in\tau_j}$$

The real equivalent follows after substituting $(a_n, -b_n)/2$ for $\boldsymbol{c}_n$, assuming $\boldsymbol{F}$ is real and making some simplifications.

Case: Impulse Function

A simple example should make it clear how to use the above definitions to translate a tabular function to Sage inputs. In this case, the tabular function is the impulse function which takes the value N at $\tau_0 = 0$ and zero for all other τ_j. When plotted out, the impulse function looks like this:



Since this is an even function ($f(\tau_j) = f(\tau_{N-j})$), only the cosine coefficients are nonzero. And even then, most of the terms in the above summations drop out because $f(\tau_j) = 0$ for $j \geq 1$. So with a little thought, it is clear that the expansion for the impulse function for the case of N (NTnode) even is

$$f(\tau_j) = 1 + 2\cos\tau_j + 2\cos 2\tau_j + \ldots + 1\cos N/2\,\tau_j$$

The impulse function phase-shifted by α would be

$$f(\tau_j) = 1 + 2\cos(\tau_j + \alpha) + 2\cos(2\tau_j + 2\alpha) + \ldots + 1\cos(N/2\,\tau_j + N/2\,\alpha)$$

For the case of N odd, the coefficient of the last term would be 2, rather than 1 and its order would be $(N-1)/2$, rather than $N/2$.

9.6 Data Pairs

Data-pair variables have a multi-line display like this:

```
ExtrmT        temperature extremes (NonDim, K) data pairs...
  ( 0.000E+00,  3.000E+02)
  ( 2.500E-01,  4.000E+02)
  ( 1.000E+00,  2.000E+02)
```

Within the parenthesis, the left number represents the independent variable and the right number the dependent variable. In this example we have the extreme temperature values occurring at dimensionless positions 0, 0.25 and 1.0, respectively. Generally speaking, data-pair variables are used as outputs.

Data-pair variables are not directly referenceable in user-defined algebraic expressions. However for a data-pair variable X, the following subfields are referenceable:

X.TData.n	nth independent value in X's defining data
X.FData.n	nth dependent value in X's defining data

where n ranges from 1 to the number of interpolation pairs defining X. For example ExtrmT.FData.2 in the above example would be 4.0E2.

The notation TData and FData to denote independent and dependent values requires some explanation. Sage's original purpose for data pairs was to store physical properties $F(T)$ as a function of temperature T. In that context the notation TData and FData makes perfect sense. Later on data pairs were used to represent other independent variables as well but the designation TData for the independent value was retained.

9.7 Cubic Splines

Cubic-spline variables are similar in outward appearance to data-pair variables. They have a multi-line display like this:

```
Tinit         initial temperature (NonDim, K) unit spline..
  ( 0.000E+00,  3.000E+02)
  ( 5.000E-01,  6.000E+02)
  ( 1.000E+00,  9.000E+02)
```

What you see are interpolation pairs through which a cubic-spline curve will be fit, with the left number representing the independent variable and the right number the dependent variable. In this example we have the specifications for a cubic spline passing through the temperatures 300K, 600K and 900K at dimensionless positions 0, 0.5 and 1.0, respectively. The number of interpolation pairs is arbitrary so long as there are at least two. Changing them is easily done via dialog box commands.

Generally speaking, cubic spline variables are used to specify things like initial temperature profiles or interpolation data for material properties. The independent variable may be dimensionless, as in this example, or itself a dimensioned quantity. This is generally clear from context. Further information for

specific variables can be found in the documentation for the model component in which it resides.

There are variations of cubic splines for data that vary over several orders of magnitude, such as is the case for most material properties. The log-log spline, for example, log-transforms both independent and dependent variables before curve fitting — as if curve-fitting to the data on a log-log plot. The so-called semi-log variation log-transforms only the dependent variable. The use of un-transformed vs. log-log or semi-log splines is entirely transparent to the user.

Cubic-spline variables are not directly referenceable in user-defined algebraic expressions. However for a cubic-spline variable Y, the following subfields are referenceable:

Y.TData.n	nth independent value in Y's defining data
Y.FData.n	nth dependent value in Y's defining data

where n ranges from 1 to the number of interpolation pairs defining Y. For example Tinit.FData.2 in the above example would be 6.0E2;

You may also reference the interpolated value of a cubic-spline variable by following the variable name with an argument enclosed in parenthesis. For example, if Y is a cubic-spline variable then Y(0.25) returns Y interpolated at $x = 0.25$, where x is the independent variable. The argument can be any valid expression as defined in chapter 10, not just a simple constant. The dimensional units of the argument are presumed to be the same as the current dimensional units in effect for the independent variable (Options|Model Class dialog).

9.8 Enumerated

Enumerated variables display like this:

```
Solid           wall material                   SS304...
```

where the identifier on the right is one of several discrete alternatives selected in a dialog box. If you select the "name only" display option within the Sage options dialog (Options|Sage menu command) this is all you will see. If you select the "full detail" display option, enumerated variables are followed by additional display lines containing further information. For example, the above variable is followed by

```
Density =  7.800E+03
(T, Ks(T))...
( 2.000E+00,  1.400E-01)
( 2.000E+01,  2.000E+00)
( 2.000E+02,  1.400E+01)
( 1.500E+03,  3.000E+01)
(T, Cs(T))...
```

```
( 1.000E+00,  4.600E-01)
( 1.000E+01,  4.900E+00)
( 3.000E+01,  2.900E+01)
( 8.000E+01,  2.100E+02)
( 3.000E+02,  5.000E+02)
( 1.000E+03,  6.100E+02)
( 1.500E+03,  6.600E+02)
```

which define its density and cubic-spline interpolation pairs used for its conductivity and specific heat. You cannot directly change any of this defining data from within Sage. If you want to do this you must create a new enumerated variable instance with some unique identifier name and add it to the data file from which Sage reads its enumerated variables. The enumerated-variable data file you will have to modify depends on the particular type of enumerated variable you are concerned about. More information on this topic should be available in documentation for the particular model class you are working with.

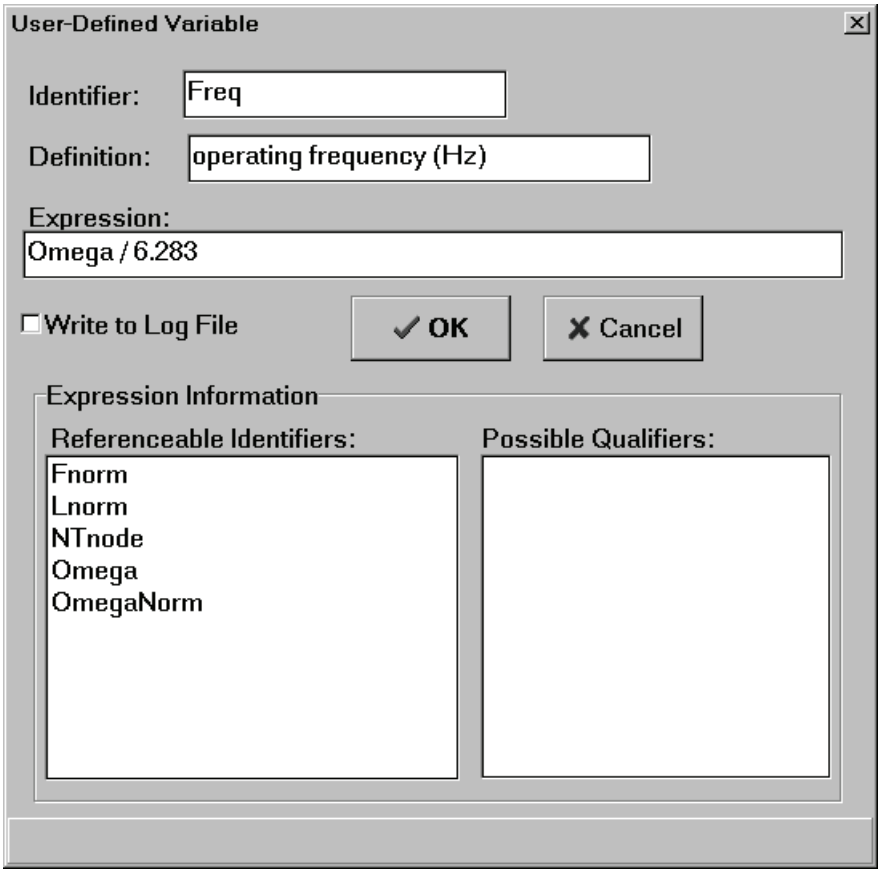
You may not reference enumerated variables in user-defined algebraic expressions. Nor may you reference any of the component variables defined within.

Chapter 10

Entering Algebraic Expressions

Algebraic expressions are the common element of constraints, objective functions and user-defined variables. You enter them as ordinary strings of text within Windows edit controls. Then, at the appropriate time, Sage reads and interprets (parses) them and converts them to meaningful calculations. A typical dialog box in which you are asked to enter an expression is the one for

user-defined variables which looks like this:



In all cases, the expression you enter is the basis for Sage to calculate some new value in terms of other variable values in the model. But there are rules of syntax to follow.

Programmers will have an advantage here because the expressions understood by Sage follow the common rules of syntax for most programming languages — particularly Pascal. For example, if `A`, `B`, and `C` are valid variable identifier names, then the following are all valid expressions:

```
A + B - C
A + 2*B
((A+B)/C + 3.0E-2) / 5.0
```

Of course, most variable names are longer than a single character.

Sage does not get around to actually parsing your expression until you tell it to, by selecting the Process|Solve, Process|Parse Solution, Process|Optimize or Process|Parse Optimization menu commands. This allows you to reference identifiers not yet created and otherwise get away with anything in your expression.

If, after you have entered your expression and closed its dialog box, you want Sage to check it for errors, just select Process|Parse Solution in the case of a user-defined variable or Process|Parse Optimization in the case of a constraint or objective function. These commands are equivalent to the *compile* step in a conventional computer language. If you are pretty sure your expressions are correct you can omit this step and proceed directly to Process|Solve or Process|Optimize. In this case, compiling, if successful, will be immediately followed by execution.

10.1 Ground Rules

Sage expressions are not case sensitive. That is, upper and lower case letters are equivalent within identifiers or constants. (Although there is only one valid letter for constants: `E` or its lower-case equivalent `e` used in an exponential format like `1.0E-2`).

Generally speaking, Sage ignores blank spaces, whether leading, trailing or embedded within your expression. You are welcome to use them, though, to separate variable identifiers from numerical constants or operators, thereby making your expressions more readable. However, blank spaces do connote separation. So you must not embed them within identifiers or constants, otherwise Sage will misunderstand your meaning.

10.2 Identifiers

The identifiers used in expressions are the variable names already in use in the model or those you create yourself for user-defined variables. All must begin with a letter, followed by one or more letters or digits. The following are all valid identifiers

```
x
Y1
Aardvark
PowerOutput
```

Some variable types have sub-fields that are accessed by appending a period directly after an identifier name, followed by the appropriate qualifier. For example, if `Aardvark` is a phasor-type variable, then

```
Aardvark.amp
Aardvark.arg
```

are valid identifiers. `Aardvark` alone, although valid as an identifier, will generate an error at compile time because it cannot be resolved into a floating-point type. See chapter 9 for the subfield qualifiers for the various types of variables.

10.3 Operators

The list of valid operators is

`+`	add
`-`	negate or subtract
`*`	multiply
`/`	divide

Parenthesis may be used in the usual way to control operator precedence, and they may be nested to any level. Lacking parentheses, operations occur in the usual pecking order. That is, multiplication and division come before addition and subtraction. For example, `(3 + 4 / 2)` evaluates to 5 not 7/2.

10.4 Constants

Numerical constants are also allowed in expressions. They are always understood as floating-point numbers, rather than integers. The following are all valid representations for the same constant:

```
234
2.34E+02
234.0
```

Constants may not begin with a period. In other words, .234 is not a valid constant. The reason for this rule is to avoid confusion with periods as subfield qualifiers in identifiers.

10.5 Built-In Functions

Sage has a number of built-in functions you can use in expressions. These require either zero, one or two arguments following the function identifier, enclosed within parentheses. In the following table, the first argument is indicated by `x`, the second by `y` and either one may be another expression, perhaps involving a nested function. For example `Sqrt( 10 * Abs( ArcSin(Pi) ) )` is a legitimate construct.

function	meaning	restrictions
`Pi`	π (3.14159...)	
`Abs(x)`	absolute value of x	
`Sqrt(x)`	square root of x	$x \geq 0$
`Sqr(x)`	square of x	
`Exp(x)`	exponential function of x	
`Ln(x)`	natural logarithm of x	$x > 0$
`Sin(x)`	sine of x	
`ArcSin(x)`	inverse sine of x	x in $[-1, 1]$; result in $[-\pi/2, \pi/2]$
`Cos(x)`	cosine of x	
`ArcCos(x)`	inverse cosine of x	x in $[-1, 1]$; result in $[0, \pi]$
`Tan(x)`	tangent of x	
`ArcTan(x)`	inverse tangent of x	result in $[-\pi/2, \pi/2]$
`Sinh(x)`	hyperbolic sine of x	
`ArcSinh(x)`	inverse hyperbolic sine of x	
`Cosh(x)`	hyperbolic cosine of x	
`ArcCosh(x)`	inverse hyperbolic cosine of x	$x \geq 1$; result ≥ 0
`Tanh(x)`	hyperbolic tangent of x	
`ArcTanh(x)`	inverse hyperbolic tangent of x	result in $(-1, 1)$
`Max(x, y)`	larger of x and y	
`Min(x, y)`	smaller of x and y	
`Power(x, y)`	exponentiation (x^y)	$x > 0$
`Amp(x, y)`	magnitude of complex number (x, y)	
`Arg(x, y)`	argument (phase) of complex number (x, y)	result in $[-\pi, \pi]$

The arguments of the trigonmetric functions are in radians. The `Power` function is mainly intended for raising positive arguments to fractional powers. For polynomial expressions using integer powers it is faster and less restrictive (can use a negative `x`) to use direct multiplication, perhaps in conjunction with the `Sqr` function — for example, `x*Sqr(x)` to represent x^3.

10.6 Model-Specific Functions

In addition to the built-in functions, any referenceable cubic-spline or Fourier-series variable in your model may also be used as a function by following its identifier name with an argument enclosed in parentheses. For example, if `Area` is a cubic-spline variable, then

```
Area(0.5)
```

is a valid expression that returns the interpolated value of `Area` at $x = 0.5$, where x is the independent variable. The argument can be any valid expression, not just a simple constant.

In general, the units of the argument are dimensional according to the units in effect for the independent-variable part of the underlying cubic-spline or

Fourier-series variable. In the case of Fourier-series variables, the argument is always an angle with units of radians or degrees as specified in the Options|Model Class dialog. In the case of cubic-spline variables, the argument is often dimensionless but not always. If not dimensionless, its units are listed after the variable definition in the display window or output listing. Argument units may matter if you specify an argument as a simple constant then change the units setting that applies to the argument. For example, if `P` is a Fourier-series variable then `P(45)` will mean one thing if the units for the argument are degrees and quite another if radians.

10.7 Referenceable Variables and Qualifiers

You may not just include any old identifier name in the expression you enter. It must be the name of a referenceable identifier. That is, an identifier known to exist at the current level within the model structure and, if it is not a real or integer type variable, with potential to be converted to a floating-point value through a subfield qualifier, as noted above. For your convenience, identifiers for all currently referenceable variables are presented in a list box at the bottom left of any dialog requiring you to enter an expression. Clicking on an identifier in the list will display its possible subfield qualifiers, if any, in the box to the right (*see* the above illustration). You cannot drag and drop identifiers and qualifiers into your expression. You must type them in, with a period separating the two (if a qualifier is required). Be alert for qualifiers requiring sub-qualifiers, such as most Fourier-series qualifiers and cubic-spline qualifiers as explained in chapter 9. Sub-qualifiers are generally integer indices and you must remember to type them in yourself.

The rule about only using referenceable identifiers from the list box is not a hard and fast rule. You may reference other identifiers in your expression as well, if you want. But before you actually compile your expression you must create a user-defined variable having that name and whose scope includes the model level in which your expression resides.

10.8 Variable Scope

The default scope of a variable is the model component in which it resides and all generations of its child models. Looking at it another way, you may reference from a child model component any variable defined in a Parent model, or an ancestor any number of generations back. But you cannot reference the other way. At least not usually. The reason for this is to prevent variable name conflicts from the point of view of a parent model component looking at two or more children with the same variable identifier.

There is a way around this restriction, though, in the case of user-defined variables. For these, the default variable scope applies at the moment of creation, but may be manually extended to include any number of generations back

by clicking on the Increase button located at the bottom of the Specify|User Variables dialog box. The so-called *export level* may be reduced again by clicking on the Decrease button. This scope extending feature is very useful when you need to compute *big-picture* quantities in terms of variables localized in a number of distinct model components. You simply export the various local variables to some common level, say the root model, then reference them there in constraints, the objective function or other user-defined variables.

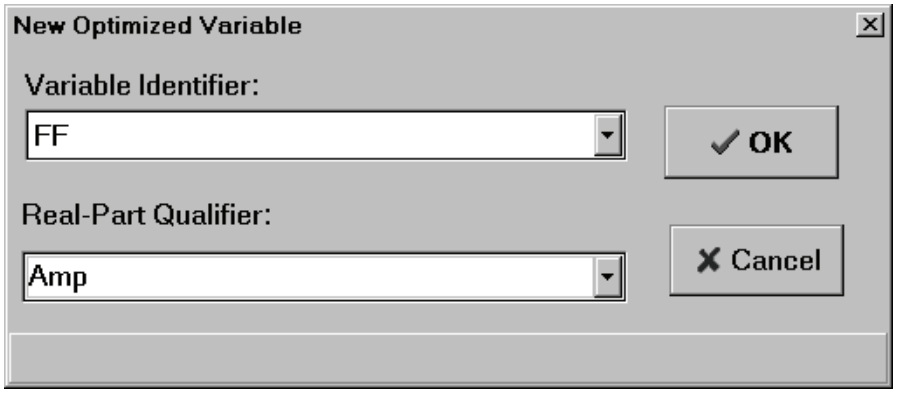
One final note: problems of self-reference will cause a runtime error as the numerical co-processor stack overflows. Self reference will occur if a variable named `A` references a variable named `B` in its defining expression while at the same time `B` references `A`. While such referencing may be allowed in computer languages that assign values in strict sequential order, this does not apply to Sage.

Chapter 11

Specifying Drivable Variables

Drivable variables are those whose values are automatically assigned by a Sage driver, such as the mapper or optimizer. Generally speaking, drivable variables must be real-valued independent inputs. However, they may also be real parts of certain complex (phasor), splines or Fourier-series variables, provided they are independent inputs. Dependent (output) variables are not drivable, for obvious reasons. Nor are constants (certain inputs) because of assumptions about constants made by the optimizer – namely, that they be constant.

Drivable variables are specified by their identifier name along with an optional subfield qualifier, similar to the way variables are referenced in algebraic expressions. But when specifying drivable variables there is no need to manually type in identifier + qualifier. You just select the two from combo-box lists within the selection dialog. For example, the selection dialog for an optimized variable (Specify|Optimized Variables) might look like this:



If there is no qualifier required for a selected identifier you will not even see

the qualifier list.

The above selection process applies in most cases. However, for Fourier-series or cubic-spline variables, where the qualifier requires a subqualifier, as explained in chapter 9, you must manually append the sub-qualifier to the qualifier. The sub-qualifier is generally an integer index which you type directly into the qualifier combo-box as an edit control. Be sure to separate qualifier from sub-qualifier with a period.

Chapter 12

Resonant-System Model Class

A good way to get acquainted with Sage is by running some simple spring-mass-damper resonant system models. This is the purpose of the so-called gizmo model class. You run gizmo.exe by clicking on the Resonant System icon in the Sage program group window, which launches the program.

A typical thing you might do with a gizmo-class spring-mass-damper system is specify a fixed-frequency forcing function as an input and solve for response amplitude of a reciprocating mass as output. There are other things you might do as well, depending on the individual model components you select into your model. Generally speaking, the main model components within gizmo are variations of *moving parts*, a general class including reciprocating masses as well as various types of springs and dampers.

Most gizmo model components come in both phasor and time-ring varieties. In either case, the motion is assumed periodic in time. But in the phasor case, motion and forces are assumed sinusoidal so they can be represented by their complex amplitudes (see chapter 9), while in the time-ring case, model variables are solved on a periodic time-grid — a so-called time ring.

12.1 Boundary Connections

Gizmo model components communicate with each other in one of two ways. Either via forces acting on points of attachment or pressures acting on area faces. After model solution, two attachments points connected together will share the same motion, whereas two area faces connected together will share the same displaced volume, much as two pistons connected by an incompressible hydraulic circuit would. Connected faces need not have the same area. In either case there is a positive or negative orientation to the connection. Positive-facing connectors show up as labeled arrows on the right edge of model components. Negative-facing connectors show up on the left. Only opposite-facing connectors

may be connected together. This is Sage's equivalent of Newton's law, which says (sort of): a force may only mate with an equal but opposite reaction. Although the real world admits forces directed arbitrarily in three-dimensional space, gizmo models confine forces to lie along a one-dimensional axis.

In the edit form, force connectors are arrows labeled either $\mathsf{F_{phsr}}$ or $\mathsf{F_{Gt}}$, depending on whether the connecting force is a phasor or time ring. Pressure connectors are arrows labeled either $\mathsf{P_{phsr}}$ or $\mathsf{P_{Gt}}$. You are only allowed to connect a force to a force and a pressure to a pressure, and only when both have the same solution structure. That is, you cannot connected phasors to time rings.

12.2 Moving-Part Attachments

All gizmo moving parts have the ability to produce child model components representing points of attachment and area faces. When you drop a moving-part component from the gizmo component palette into the edit form, it starts out with no connector arrows whatsoever. That is, you cannot connect it to anything, initially. In order to make connections, you must first create one or more attachment child model components. To do this you first double-click the left mouse button on the model-component icon, which displays its child-model page in the edit form. Then, in the Sage component palette you will find a subset of the following point attachment and area-face attachment child model components available for dropping into the edit form:

Icon	Purpose
	phasor negative-facing force attachment
	phasor positive-facing force attachment
	phasor negative-facing area attachment
	phasor positive-facing area attachment
	time-ring negative-facing force attachment
	time-ring positive-facing force attachment
	time-ring negative-facing area attachment
	time-ring positive-facing area attachment

When you drop one of these into the edit form, it is born with a connection arrow which you can then move up one level to the parent-model page for connection there to a mate with another moving-part model component.

Generally speaking you will want to create exactly one connector child component for each moving part you want to connect to. But you must not make duplicate connections between the same moving parts. Doing so will cause a singular equation system at solving time.

This business of using child model components to customize boundary connections is a typical Sage convention for all model classes, not just Gizmos. It allows you to create exactly as many connections as your model components require without presuming any minimum or maximum amount. However, the idea may be a bit confusing to new users of Sage. Keep this in mind: Model components get boundary connectors in one of two ways. Either they are born with them right out of the palette. Or they are hidden in potential child model components in the palette of a lower-level page, there to be user-created and elevated to the proper level by mouse operations. The first behavior is appropriate when one and only one boundary connection makes sense for that type of component. In our gizmo example, exactly one force boundary connection is appropriate for a force-attachment component. The second behavior is appropriate when any number of boundary connections make sense. As is the case for the number of connectors attached to a reciprocating mass component.

12.3 Moving-Part Variables

Moving parts also have a number of output variable found in all descendants. Phasor moving-parts have variables:

F : (phasor, N) Net boundary force acting on the component resulting from all connections. A negatively directed boundary force cancels a positively directed force.

Wnet : (real, W) Time-averaged power delivered by all boundary forces. The mean value of net force times velocity.

And time-ring moving-parts have variables:

F : (Fourier series, N) Net boundary force acting on the component resulting from all connections.

W : (Fourier series, W) Power delivered by all boundary forces. Net force times velocity. The mean value is equivalent to Wnet in the TPhsrMov model component.

12.4 Generic Springs



Generic springs are moving-part descendants that come in a phasor version and a time-ring version. Both introduce a new input variable:

K : (real, N/m) Spring stiffness.



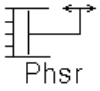
In either case, the mathematics behind the scene solves displacement x from net applied boundary force F and stiffness K using the equation

$$F = Kx \tag{12.1}$$

For the phasor spring, variables F and x are complex numbers and solution for x is straight-forward in terms of complex algebra. For the time-ring spring F and x are discrete real values on a grid and the solution is even easier.

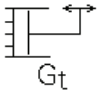
You may think of Sage springs as having one end fixed to ground and the other subject to any number of attachment points or faces for connection to other model components — generally to reciprocating-mass components. Generic springs have no mass.

12.5 Generic Dampers



Generic dampers are moving part descendants that also come in phasor and time-ring versions. Both introduce a new input variable:

D : (real, N/(m/s)) Damping coefficient.



This time, the mathematics behind the scenes solves displacement x from net applied boundary force F and damping coefficient D using the equation

$$F = D\dot{x} \tag{12.2}$$

where $\dot{x}$ is velocity. For the phasor damper, we may write $i\omega x$ for $\dot{x}$ so it is easy to solve for x explicitly. For the time-ring damper, solving for x is less direct. Time-ring moving-part model components maintain a computational grid containing, not just displacement x, but velocity $\dot{x}$ and acceleration $\ddot{x}$ as well, as individual solution variables. We may call these variables x, x_d and x_{dd}. The above equation determines x_d explicitly, time-differentiating x_d determines x_{dd} explicitly and the defining condition $\dot{x} = x_d$ determines x implicitly.

Like springs, you may think of Sage dampers as having one end fixed to ground and the other subject to any number of attachment points or faces for connection to other model components. You will usually be connecting them to reciprocating mass components. They, too, are massless.

12.6 Reciprocating Masses



Reciprocating masses, or reciprocators for short, are moving part descendants that come in both phasor and time-ring versions. Mathematically, they both solve displacement x from the net resultant force F and mass M using Newton's equation of motion

$$F = M\ddot{x} \tag{12.3}$$



where $\ddot{x}$ is acceleration. This time, F includes an internally-specified body forcing function in addition to applied boundary forces. For the phasor reciprocator, we may write $-\omega^2 x$ for $\ddot{x}$ so it is easy to solve for x explicitly. For the time-ring damper, solving for x in the grid of solution variables x, x_d and x_{dd} goes like this: The above equation determines x_{dd} explicitly, the defining condition $\dot{x_d} = x_{dd}$ determines x_d implicitly and defining condition $\dot{x} = x_d$ determines x implicitly.

One may think of Sage reciprocators as masses that move back and forth according to Newton's law of motion. For phasor reciprocators, mean position and velocity are both zero. All that matters is displacement from some fixed mean position. For time-ring reciprocators, mean velocity is zero as a requirement of periodic motion, but mean position may be nonzero, depending on possible position-dependent boundary forces or pressures.

The phasor reciprocator introduces new variables:

Mass : (real, kg) Reciprocating mass.

FF : (phasor, N) Forcing function. An applied body force specified within the model component itself rather than arising as a boundary force.

X : (phasor, m) Displacement from mean position.

The time-ring reciprocator introduces similar new variables, except Fourier series replace phasors:

Mass : (real, kg) Reciprocating mass.

FF : (Fourier series, N) Forcing function.

FX : (Fourier series, m) Displacement from mean position.

You can optimize any of the drivable real-parts of either the phasor or Fourier-series forcing function if you choose (*see* chapter 11).

12.7 Constrained Pistons



These model components descend from the reciprocator components previously discussed. Except they switch the role of forcing function and displacement, so that now displacement is the input and required forcing function, to achieve that displacement, is the solved quantity. Generally, you will only want to connect a constrained motion reciprocator to model components like springs and dampers that can accommodate the irresistible displacement provided by these components. You should never directly connect together two constrained motion reciprocators.



Mathematically, these components solve required forcing function F_f from net applied boundary force F_b, displacement x and mass M, using Newton's equation of motion

$$F_f = M\ddot{x} - F_b \tag{12.4}$$

where $\ddot{x}$ is acceleration.

The phasor constrained piston introduces new variables:

Xamp : (real, m) Displacement amplitude.

Xphase : (real, radians) Displacement phase angle.

FF : (phasor, N) Required forcing function to achieve Xamp and Xphase in light of applied boundary forces.

Displacement is given by two real variables rather than a single phasor so that amplitude or phase may be more conveniently optimized if need be. Physical displacement is

$$x = \mathsf{Xamp}\cos(\omega t + \mathsf{Xphase}) \tag{12.5}$$

The time-ring constrained piston introduces similar new variables, except Fourier series replace phasors.

FX : (Fourier series, m) Displacement from mean position.

FF : (Fourier series, N) Required forcing function.

You can optimize any of the drivable real-parts of displacement if you choose (*see* chapter 11).

12.8 Relative Moving Parts

The preceding springs and dampers are referenced to ground (fixed inertial frame) at the end opposite the connection. So, for example, there is no way to connect one between two reciprocating masses. Relative springs and dampers solve this problem. Relative springs and dampers descend from the common abstract class of relative moving parts.

Relative moving parts contain two position coordinates instead of one — the coordinates of the positive and negative boundaries. Before, all connections were thought of as made to the same boundary, which was represented by a single coordinate. Now, positive connections are thought of as made to the positive boundary and negative connections to the negative boundary.

Relative moving parts are born with exactly two force connectors, with no provision for additional connectors. You may connect these built-in connectors to opposite force connectors in any of the previous moving parts (such as reciprocating masses). In any event, you must connect them to something, lest a singular solution result.

Like the previous moving parts, relative moving parts come in phasor and time-ring variations. All phasor relative moving parts have the variables:

Fneg : (phasor, N) Force acting on the component negative boundary.

Fpos : (phasor, N) Force acting on the component positive boundary. By default, Fpos is equal and opposite Fneg, or 180 degrees out of phase. (*see* equation (12.6))

Wnet : (real, W) Time-averaged power delivered by the boundary forces. For a relative spring, this is zero. For a relative damper this is the power absorbed by the damper.

All time-ring relative moving parts have the variables:

Fneg : (Fourier series, N) Force acting on the component negative boundary.

Fpos : (Fourier series, N) Force acting on the component positive boundary. As for the phasor counterparts, Fpos is equal and opposite Fneg. But this time for each component of the Fourier series.

W : (Fourier series, W) Power delivered by the boundary forces. The mean value is equivalent to Wnet in the phasor counterpart.

Unless overridden in some future descendant class, Sage assumes all relative moving parts are massless. As such, the dynamic force balance may be written in terms of the negative and positive boundary forces as

$$F_{neg} + F_{pos} = 0 \tag{12.6}$$

Sage uses this condition to implicitly solve the negative coordinate x_{neg} for all relative moving parts.

12.9 Relative Springs



Relative springs, just like generic (absolute) springs, are massless springs defined in terms of the input variable:

K : (real, N/m) Spring stiffness.



Sage solves the negative coordinate x_{neg} as above and the positive coordinate x_{pos} from the applied boundary force F_{pos} and stiffness K using the equation

$$F_{pos} = K(x_{pos} - x_{neg}) \tag{12.7}$$

12.10 Relative Dampers



Relative dampers, just like generic (absolute) dampers, are massless dampers defined in terms of the input variable:

D : (real, N/(m/s)) Damping coefficient.



This time, Sage solves the positive coordinate x_{pos} from the applied boundary force F_{pos} and damping coefficient D using the equation

$$F_{pos} = D(\dot{x}_{pos} - \dot{x}_{neg}) \tag{12.8}$$

Chapter 13

Behind the Scenes

Sage model components and their variables are nothing at all like the constructs found in a typical scientific programming language like FORTRAN. They are, instead, examples of software objects which may be loosely defined as encapsulated data structures with built-in methods (functions and procedures) to manipulate their data. If this has you curious then by all means read this chapter. Otherwise you may skip it. You can run Sage perfectly well without knowing anything about its computer-science architecture.

13.1 Smart Variables

A good place to start is with the fundamental building blocks of Sage models: the variables. These are much more than named memory locations. As software objects they have some useful behaviors as well. For example, they know their names. They know how to format themselves for input and output. They know when their stored value is valid, or if it is not, how to call the proper evaluation function to make it valid. And they have a context within an overall solution scheme.

All variables are created either constant, independent, dependent or implicit. A *constant* is a variable whose value you may set via menu command at any time but remains fixed thereafter. An example of a constant would be a real-valued normalization quantity. An *independent* variable is like a constant except its value may also be set by an external driver, such as the mapper or optimizer. Collectively, constants and independent variables form the inputs of a model component. A *dependent* variable is one whose value depends on other variables, possibly other dependent variables, but eventually on non-dependent variables. Sage models contain mechanisms to invalidate dependent variables when the variables upon which they depend change. After invalidation, a dependent variable re-evaluates itself the next time it is needed. It does this by calling a function, whose address is stored locally within the variable, and waiting for the return value. An executing evaluation function may, in turn, call

upon other variables, some of which may turn out to be other invalid dependents. This process may go on for a while, cascading throughout a vast network of variables, until eventually the first evaluation function returns. In a sense, dependent variables struggling to keep themselves valid is what Sage models are all about. Finally, an *implicit* variable is one whose value is set by a nonlinear equation solver on the basis of zeroing a real-valued function whose address is, again, stored locally within the variable. Like independent variables, implicit variables are real valued. The implicit-variable evaluation function is much like a dependent-variable evaluation function, calling upon other variable values as needed, possibly giving more grief to the dependent variables who are constantly having to re-evaluate themselves. Collectively, the dependent and implicit variables form the outputs of a model component, although some are invisible to the user.

13.2 Smart Models

As you may have surmised by now, models in Sage are just collections of smart variables organized to exhibit some required behavior. But specific behaviors pertaining to physical phenomena occur in the high-level cerebral cortex of a model. Deep in the reptilian brain stem, common to all models, are a number of fundamental behaviors.

At the lowest level of awareness, models maintain a number of data structures which help them to manage their variables and any child models they might have. Thus they are able on demand to load and store themselves, their variables and their children to and from a disk file. They know how to invalidate any of their dependent variables currently valid and which models are affected by their non-dependent variables so they may ask them, in turn, to invalidate their dependent variables when required. And they are able to interface with Sage's I/O routines, its mapping and optimization drivers and its nonlinear solver in order to count, collect together or otherwise deal with variables that are either inputs, outputs, mapped, optimized or solved.

13.3 Grids

At a slightly higher level of awareness, some models know how, when asked, to deal with the various computational grids common to numerical analysis. There is a split in the model evolutionary tree here with one branch knowing about grids and the other not. The knowing branch is further divided according to the type of grid it understands. Presently supported are one-dimensional spatial grids, periodic one-dimensional time rings and two-dimensional space-time grids, denoted respectively by G_x, G_t and G_{xt}. All grids maintain nodes of real-valued variables (smart ones) at discrete spatial positions or times and support the common operations on grids such as numerical differencing, integrating and averaging. A periodic time ring is a grid where the nodes may be thought of as

equally spaced on the perimeter of a circle, so that there is no actual first or last node, except as a computer-storage convenience. Time rings are appropriate for modeling time-periodic phenomena.

13.3.1 Spatial Grids

Spatial grids are compatible with staggered-grid, also known as control-volume, solution methods. That is, they may be thought of as containing an integer number of control volumes or cells, with nodes falling at the boundaries and centers of each cell, so the total number of nodes is always odd. They do spatial differencing according to the central formula

$$\frac{\partial f}{\partial x}(i) = \frac{f(i+1) - f(i-1)}{2\,dx} \tag{13.1}$$

where i is the spatial index. For integration, spatial grids use a variation of Euler's rule, sampling only every other spatial index, at cell midpoints, thereby guaranteeing a global conservation property with staggered-grid solution schemes. This follows because when one integrates a term $\frac{\partial f}{\partial x}$ across a computational domain there is a pair-wise cancelation of f values, except at the two endpoints. In physical terms the global conservation property may be stated as "what flows out of one computational cell enters the next". The reason higher order differentiation is not used is that it would destroy this global conservation property, or complicate it greatly.

Spatial grids interpolate between neighboring solved values using Lagrange polynomial interpolation. Staggered-grid solutions generally alternate between solved and interpolated values. In the case of interpolation there is no global conservation property to worry about so, in principle, the order of interpolation can be as high as required. The current limit is set to cubic (third order) because of concerns that higher order will only cause numerical trouble due to the tendency of higher-order polynomials to produce wildly oscillating values between tabulated values. The order of interpolation may be set in the Options|Model Class dialog as either linear or cubic for those models using grids.

When possible, the neighboring solved values used in interpolation are distributed equally on either side of the interpolation point with the interpolation point at the center. For example, linear interpolation amounts to the averaging the values of the surrounding two solved variables.

$$f(i) = \frac{f(i-1) + f(i+1)}{2} \tag{13.2}$$

Cubic interpolation involves the weighted average of four neighboring values.

$$f(i) = -0.0625f(i-3) + 0.5625f(i-1) + 0.5625f(i+1) - 0.0625f(i+3) \tag{13.3}$$

When neighboring values fall outside the computational domain they are shifted toward one side or the other, as required. For example, linear interpolation at

the negative and positive domain endpoints become

$$f(i) = \frac{3}{2}f(i+1) - \frac{1}{2}f(i+3) \tag{13.4}$$

$$f(i) = \frac{3}{2}f(i-1) - \frac{1}{2}f(i-3) \tag{13.5}$$

unless the computational domain contains only one cell, in which case nodes $i+3$ or $i-3$ lie outside the computational domain and the interpolated value is just the cell-center value.

In general, spatial interpolation is carried out according to the formula

$$f(x_i) = \sum_{k=1}^{N} A_k f(x_p) \tag{13.6}$$

where N is an even number corresponding to the number of points in the set of neighboring solved points x_p used for the interpolation. The neighboring points x_p are a set of consecutive staggered locations in the neighborhood of the point of interpolation x_i, with index p given by

$$p(k) = (i-1) + 2(k - k_i) \tag{13.7}$$

k_i is the offset (0 to N) of the point of interpolation relative to the x_p. $k_i = 0$ corresponds to the x_p beginning at x_{i+1} (i.e. interpolating at the negative endpoint), $k_i = N/2$ corresponds to the x_p distributed equally on either side of x_i (i.e. central interpolation) and $k_i = N$ corresponds to the x_p ending at x_{i-1} (i.e. interpolating at the positive endpoint). The coefficients A_k are tabulated depending on N and k_i using a variation of Neville's recursive algorithm, as follows. Start with

$$F_{l,0} = \delta_l^k \qquad l = 1, \ldots, N \tag{13.8}$$

where δ_l^k is the Kronecker delta function (0 if $l \neq k$, 1 if $l = k$). Then compute

$$F_{l,m} = \frac{D_l F_{l+1,m-1} - D_{l+m} F_{l,m-1}}{2m} \qquad \begin{matrix} m = 1, \ldots, N-1 \\ l = 1, \ldots, N-m \end{matrix} \tag{13.9}$$

where the D_l are the offsets of the lth p index relative to i.

$$D_l = 2(k_i - l) + 1 \tag{13.10}$$

and finally

$$A_k = F_{1,N-1} \tag{13.11}$$

This requires some explanation. Neville's recursive algorithm (*see* section 3.1 of [3]) obtains the unique $N-1$ order polynomial approximation at an interpolation point x_i to the function with values f_p at the points x_p. If the point x_i and the x_p always remain the same, the interpolation may be represented in the form of equation (13.6) where the A_k values need to be calculated only once and then saved for subsequent use. The way to calculate the A_k's is to trace

the contribution of each f_p component through the interpolation process. This is easily done by applying the Neville algorithm with all f_p values zero except for $f_p = 1$ for p corresponding to the index k. This is exactly what the above algorithm does by interpolating the set of basis functions δ_l^k. The one confusing point is that for purposes of the algorithm the tabulated x_p and f_p points are indexed sequentially using what is denoted above by the l index, as $x_{p(l)}$ and $f_{p(l)}$ for $l = 1, \ldots N$.

13.3.2 Time Grids

Time grids, or rings in Sage parlance, assume a periodic solution. They do time differencing using a scheme with its roots in the three-point backward differencing method attributed to Richtmyer and Morton [4]. Letting j denote the time index, this method is

$$\frac{\partial f}{\partial t}(j) = \frac{3f(j) - 4f(j-1) + f(j-2)}{2\,dt} \tag{13.12}$$

While this is a starting point for the method in Sage, it is not the actual method used. The method was originally improved by re-deriving it to "get the right answer for" or annihilate periodic sinusoidal functions rather than the first few terms of a polynomial, as the method was originally conceived. This was merely a matter of replacing the integer coefficients in the above formula with coefficients β_0, β_1, β_2 solved to annihilate test functions $f = 1$, $f = \sin$ and $f = \cos$. The result was a three-point backward differencing formula that served Sage well for several years.

But the problem was that although the method gave exact results for time-derivatives of purely sinusoidal functions, it introduced progressively increasing errors for higher harmonics, depending on the number of nodes in the grid. So the method was extended to annihilate as many harmonics as possible, by increasing the number of backward sampling points, according to the formula

$$\frac{\partial f}{\partial t}(j) = \sum_{k=0}^{2M} \beta_k f(j-k)) \tag{13.13}$$

where M is the order of the highest harmonic annihilated — in practice limited to $(N-1)\mathrm{div}\,2$, where N is the number of nodes in the grid. ("div" is the divide operator that rounds down to the next lowest integer) With this value of M, all available harmonics are annihilated when N is odd and all but the highest when N is even.

By requiring that formula (13.13) hold for the first M Fourier-series basis functions, it is straight-forward to show that the required β_k coefficients must

solve the following linear equation system

$$\begin{bmatrix} \sin\Delta\tau & \sin 2\Delta\tau & \dots & \sin 2M\Delta\tau \\ 1-\cos\Delta\tau & 1-\cos 2\Delta\tau & \dots & 1-\cos 2M\Delta\tau \\ \sin 2\Delta\tau & \sin 4\Delta\tau & \dots & \sin 4M\Delta\tau \\ 1-\cos 2\Delta\tau & 1-\cos 4\Delta\tau & \dots & 1-\cos 4M\Delta\tau \\ & \vdots & & \\ \sin M\Delta\tau & \sin 2M\Delta\tau & \dots & \sin 2M^2\Delta\tau \\ 1-\cos M\Delta\tau & 1-\cos 2M\Delta\tau & \dots & 1-\cos 2M^2\Delta\tau \end{bmatrix} \begin{bmatrix} \beta_1 \\ \beta_2 \\ \beta_3 \\ \beta_4 \\ \vdots \\ \beta_{2M-1} \\ \beta_{2M} \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \\ -2 \\ 0 \\ \vdots \\ -M \\ 0 \end{bmatrix} \tag{13.14}$$

with $\Delta\tau$ the dimensionless time step

$$\Delta\tau = \frac{2\pi}{N} \tag{13.15}$$

and the zeroth coefficient computed explicitly as

$$\beta_0 = -\sum_{k=1}^{2M} \beta_k \tag{13.16}$$

once those for $k \geq 1$ are available. This implicit formulation is no problem computationally. It just means that Sage must invoke a linear solver object to compute the β_k's, rather than evaluate some, perhaps, complicated expressions involving sine and cosine functions. This is required only once upon creation of the computational grid.

For integration, time grids use Euler's rule without modification — sampling and giving equal weight to every time index. This is the only possibility, since logically time grids have no beginning or end indices, or any other basis, to distinguish one node from another for weighting purposes.

13.4 Connecting Things Together

So just what it does it mean to connect two model components together across a boundary? And how is it that information passes between the two components? Whenever you make a connection there is actually a third invisible object created, a boundary connector. This object is a close relative of a model component in that it is a collection of smart variables organized for a specific behavior. The behavior in this case is to provide a common boundary-value variable (or variables) to be used by the adjoining models as part of their solution and to receive from those models, in exchange, sufficient information to establish the validity of the boundary variable. This information generally takes the form of some quantity that must be conserved across the connection.

Take steady heat flow for example. We might imagine two thin heat-conducting rods connected in series between an isothermal source and an isothermal sink. Each rod is a separate model component and the connection in the middle is the current point of interest. The physical principle governing heat flow is this: Heat flows across the connection until the rod endpoint temperatures are equal.

Simple. The Sage equivalent goes like this: Heat flow is an implicit variable managed by a boundary-connector object in the middle. Each rod maintains its own independent solution, producing an axial temperature distribution as a function of connector heat flow. When the user connects the two rod ends together, each rod receives a pointer to the connector object which it can call upon to read the current value of heat flow. Meanwhile, the connector object receives pointers to the appropriate endpoint-temperature variables within both rod models. It evaluates the difference of these two temperatures as the to-be-zeroed function component associated its implicit heat flow variable. A nonlinear solver object in the background sees all this as just another equation to be solved in its equation system, and iterates the value of heat flow until temperature continuity is achieved. This is essentially how most Sage connector objects work.

As a matter of notational convenience, we could denote the previous heat-flow boundary-connector example as $Q(T)$ where Q represents the implicit heat-flow variable in the connector and T represents endpoint temperatures in the adjoining rods, continuity of which determines Q. Using this notation we can quickly summarize the basic types of boundary connectors available in Sage:

$F(\Delta X)$ — Force determined by positional displacement continuity.

$P(\Delta V)$ — Pressure determined by volumetric displacement continuity.

$Q(T)$ — Heat flow determined by temperature continuity.

$\rho(P)$ — Density determined by pressure continuity.

$\dot{m}(s)$ — Gas mass flow rate determined by state-variable continuity. Actually, $\dot{m}$ symbolizes combined mass flow rate, mass-density jump and energy-density jump, while s symbolizes, momentum flow, energy flow and a momentum equation.

Not very many connector types, really. However, as with models, connectors are further subdivided according to the numerical representation of the solved variables within themselves and in the models to be joined. For each basic type there may be variations for steady-valued solutions, complex or phasor solutions and the various types of grid solutions. But the basic types of boundary connectors are those listed.

13.5 Solving

When you elect to solve the model hierarchy you have specified, you are really calling into play another software object: a nonlinear solver. The purpose of this object is to orchestrate the iterative solution of a large system of nonlinear equations — one equation (setting the evaluation function to zero) for each implicit variable in the model hierarchy. Although often numbering in the hundreds, these implicit variables are generally invisible to the user. They come about automatically, as computational grid variables and the like.

The nonlinear solver does its job by solving a sequence of linear approximations to the nonlinear equations. The coefficients of the linearized equations result from numerical partial derivatives of the evaluation functions taken with respect to the model's implicit variables. The result is a sparse matrix, solved with a special sparse-matrix solver. The solution becomes a search direction along which to seek the nonlinear solution. This process repeats until the nonlinear model equations are all satisfied within some prescribed tolerance.

Mathematically, each iteration takes the form of solving the equation

$$\boldsymbol{J}\Delta V = -F \tag{13.17}$$

for the step ΔV, where $\boldsymbol{J}$ is the Jacobian (partial-derivative) matrix and F is the to-be-zeroed function vector. This is Newton's method. While Newton's method works well for *nearly* linear system functions, it can fail to converge for nonlinear ones, especially when the starting value for V is far from the final solution. In particular, discontinuities of implicit-variable initial values across connections have been observed to be a chief cause of non-convergence in Sage. The implicit function components corresponding to these discontinuities are of the form $F = V_+ - V_-$, where V_+ and V_- are the implicit variable values on either side of the connection. Newton's method tries to zero the discontinuity in one step, which tends to destabilize the solution. It is not too difficult to avoid this problem by relaxing variable discontinuities more slowly, which is what Sage's nonlinear solver does.

The idea is to replace F on the right-hand side of (13.17) by ΔF, the desired change in F. For most components, ΔF_j is just $-F_j$, meaning that F_j is allowed to step all the way to zero. But for key components (those of the form $F = V_+ - V_-$), ΔF_j is some smaller amount — small enough to avoid destabilizing the solution. The maximum allowable ΔF_j is something that is left to smart variables to decide.

13.6 Optimizing

Likewise, when you elect to optimize your model, you are calling into play yet another software object: an optimization driver. The nonlinear optimizer in Sage employs a variation of Powell's sequential-quadratic-programming method [2] which locally approximates the nonlinear optimization problem by a succession of quadratic sub-problems (quadratic objective function and linear constraints) each of which is readily solved. The idea is that by doing this often enough and searching along the direction from where you are to where the quadratic minimizer lies, you will eventually converge to the actual nonlinear minimizer — or rather *a* minimizer if there happens to be more than one. Powell's algorithm builds up its quadratic programming problems as it steps along by accumulating second derivative information about the objective function and constraints. It then turns to a separate quadratic optimizer for the sub-problem solution — in the Sage implementation, a convex method suited to Powell's method, reported by Goldfarb and Idnani [1]. Complicated as this may seem, it really does work.

13.7 Delphi Spoken Here

Sage is a Delphi application (Borland International, Inc.) running under Microsoft Windows. So why not C++, some might ask, which seems to be the current language of fashion, at least in the United States. Partly for continuity's sake, since the solving and optimization engines of Sage were originally written in Pascal (the underlying language of Delphi). But also, Delphi's visual component library has taken much of the drudgery out of programming the Windows interface, a considerable part of Sage. And the inherent structure and readability of the underlying Pascal language seem to make for more reliable, maintainable code.

At any rate, the object-oriented philosophy pervades Sage. Almost everything is an object of one sort or another. This makes some things easier than before, some things harder. Easier are creating and maintaining reliable models for complicated phenomena. A new model can inherit its basic behaviors from an earlier model then add new behaviors as needed. Thus, it is straight-forward to create from a generic gas domain, three variations each with its own turbulence-transition model. Debugging is easier too, because it is a snap to isolate and test model components one at a time. Harder is realizing global connectivity from a bunch of separate objects. Seemingly little things like summing masses of different components or imposing dimensional stack-up constraints can be vexing problems in the object-oriented world. After all, objects are supposed to be self contained entities so the use of good old global variables is forbidden. In answer to this, we have such things as connector objects, which were expensive in programmer time to conceive of and implement, but promise much greater flexibility and ease of use for the ultimate user.

Bibliography

[1] D. Goldfarb, A. Idnani, *A Numerically Stable Dual Method for Solving Strictly Convex Quadratic Programs*, Mathematical Programing, **27**, pp. 1–33, (1983)

[2] M. J. D. Powell, *A Fast Algorithm for Nonlinearly Constrained Optimization Calculations*, in: *Lecture Notes in Mathematics*, 630, Numerical Analysis (Proc. Biennial Conf. at Dundee, 1977), Springer-Verlag (1978)

[3] W.H. Press, B.P. Flannery, S.A. Teukolsky, W.T. Vetterling, *Numerical Recipes*, Cambridge, (1986)

[4] R. D. Richtmyer and K. W. Morton, *Difference Methods for Initial Value Problems*, Interscience, New York, (1967)

Index

D, 73, 76
FF, 74, 75
FX, 74, 75
Fneg, 75
Fpos, 75
F, 72
K, 72, 76
Mass, 73, 74
Wnet, 72, 75
W, 72, 75
Xamp, 74
Xphase, 74
X, 74

complex variables, 49
computational grids, 78
connections, 45
    arrows, 43
    changing level, 45
    meaning of, 82
    structure, 6
    types, 83
constants, 26
constrained piston, 74
constraints
    active, 35
    infeasible, 39
    overdetermined, 39
    specifying, 35
cubic splines, 55

damper, 73
    relative, 76
data pairs, 54
diagnostics
    optimizer, 38
    solver, 26
dimensional units, 25
disk files, 4, 21
display
    add page, 14
    print, 15
    remove all, 15
    remove page, 15
    show window, 14
display form, 23
drivable variables, 67

edit
    copy model, 15
    cut model, 15
    delete model, 16
    down connector, 15
    paste model, 16
    print, 16
    show window, 15
    up connector, 15
edit form, 21, 43
    levels, 44
    palette, 44
enumerated variables, 56
    changing, 57
export level, 65
expressions, 59
    built-in functions, 62
    case, 61
    compiling, 61
    constants, 62
    identifiers, 61
    model-specific functions, 63
    operators, 61
    parenthesis, 62
    parsing, 28, 59
    qualifiers, 64

scope of variables, 64
self referencing, 65
spaces, 61
syntax, 60
variables, 64

file
listing preview, 14
new, 13
open, 13
print listing, 14
save, 13
save listing, 14
save solution grid, 14, 23
save tagged variables, 14
save-as, 13
force connections, 69
Fourier series, 51
accuracy, 52

gizmo, *see* resonant system
grids, 78
full-harmonic differencing, 81
saving to file, 14, 23
spatial, 79
time ring, 69, 81

icons
positioning, 44
initialization files, 4, 21
inputs, 24
installing, 3
integer variables, 49
interpolation, 80
order, 79

listings, 6, 14, 22
log files, 31, 36

mapping, 31
menu commands, 13
model
class, 5
component, 5
activating, 44
active, 13, 22, 23
creating, 44
cut and paste, 46
editing, 43
structure, 78
tree structure, 44–46
trees, 6
views, 22
moving part
attachments, 70
variables, 72
moving parts
relative, 75

Newton's method, 84
nonlinear programming problem, 33
normalization scale factor
specifying, 40

objective function
specifying, 35
optimization, 7, 33, 84
degrees of freedom, 35
ill conditioned , 39
line search, 38
multiple extrema, 40
non convergence, 38
pseudo-Lagrangian, 36
running, 36
optimization variables
specifying, 34
step limits, 38
weakly-determined, 39
options
model class, 19
Sage, 19

palette, 44
Pascal, 85
phasor variables, 44, 50
pressure connections, 69
process
explore optimization, 18
explore user variables, 18
map, 18
optimize, 18
parse mapping, 18
parse optimization, 19

parse solution, 18, 28
reinitialize, 19
solve, 18

real variables, 49
reciprocating mass, 73
resonant system, 9, 33, 43, 69

Sage architecture, 77
scan
comments, 16
constraints, 16
input variables, 16
mapped variables, 16
optimized variables, 16
solving, 7, 25, 83
non convergence, 26
reinitializing, 26
RMS error function, 26
spatial differencing, 79
specify
commment, 17
component name, 17
constraints, 17, 35
input variables, 17
mapped variables, 17
model order, 17
normalization scale factor, 40
objective function, 17, 35
optimization variables, 34
optimized variables, 17
user variables, 17
spring, 72
relative, 76

time differencing, 81
time ring, *see* grids

units, 25
user variables, 27
copy and paste, 30
exploring, 28
export level, 28, 65
log files, 28, 31, 36

variables
complex, 49
cubic spline, 55
data pairs, 54
drivable, 67
enumerated, 56
Fourier series, 51
integer, 49
phasor, 50
real, 49
saving to file, 14
smart, 77